CareerPulse AI: AI Resume Analysis and Placement Preparation System.

Submitted In Partial Fulfillment of Requirements
For the Degree Of

Bachelor of Science
(Information Technology)

By

Sameer Kishor Patil

Roll No: 31010923062

Guide

Mr. Bilal Shaikh

Somaiya School of Basic and Applied Sciences,
Somaiya Vidyavihar University
Vidyavihar, Mumabi - 400 077

2025-26

Somaiya Vidyavihar University

Certificate

This is to certify that the project report on dissertation entitled CareerPulse AI: AI Resume Analysis and Placement Preparation System. is bonafide record of the dissertation work done by Mr Sameer Kishor Patil in the year 2025-2026 under the guidance of Mr. Bilal Shaikh Department of Information Technology and Computer Science in partial fulfillment of requirement for the Bachelor of Science degree in Information Technology of Somaiya School of Basic and Applied Sciences, Somaiya Vidyavihar University.

Mr. Bilal Shaikh Guide	Assistant Prof. Mousmi Pawar Coordinator
Dr. Swati Maurya Head of the Department	Prof. Dr. Neetin Desai Dean,SSBAS

Date:

Place: Mumbai-77

Somaiya Vidyavihar University

Certificate of Approval of Examiners

This is to certify that the project report on dissertation entitled CareerPulse AI: AI Resume Analysis and Placement Preparation Ecosystem is bonafide record of the dissertation work done by Mr. Sameer Kishor Patil in partial fulfillment of requirement for the Bachelor of Science degree in Information Technology of Somaiya School of Basic and Applied Sciences, Somaiya Vidyavihar University.

_____ _____

External Examiner /Expert Internal Examiner/ Guide

Date:

Place: Mumbai-77

Somaiya Vidyavihar University

DECLARATION

I declare that this written report submission represents the work done based on my and / or others' ideas with adequately cited and referenced the original source. I also declare that I have adhered to all principles of academic honesty and integrity as I have not misinterpreted or fabricated or falsified any idea/data/fact/source/original work/ matter in my submission.

I understand that any violation of the above will cause disciplinary action by the college and may evoke the penal action from the sources which have not been properly cited or from whom proper permission is not sought.

Signature of the Student

Sameer Kishor Patil
Name of the Student

31010923062
Roll No.

Date:

Place: Mumbai-77

Index

Sr. No	Topic	Page No.
1	**Chapter 1 Introduction**	7-11
1.1	Introduction	7
1.2	Description	8
1.3	Stakeholders	9
2	**Chapter 2 Literature Survey**	12-15
2.1	Description of Existing System	12-
2.2	Limitations of present system	13
2.3	Gantt chart	14
3	**Chapter 3 Methodology**	16-32
3.1	Technologies Used and their Description	16
3.2	Event Table	17
3.3	Use case Diagram and Basic Scenarios & Use Case Description	18
3.4	Entity-Relationship Diagram	19
3.5	Flow Diagram	21
3.6	Class Diagram	26
3.7	Sequence Diagram	28
3.8	State Diagram	29
3.9	Menu Tree	31
4	**Chapter 4 Implementation**	33-60
4.1	List of Tables with Attributes and Constraints	33
4.2	System Coding	35
4.3	Screen Layouts and Report Layouts	51
5	**Chapter 5 Analysis, Related work done (if any)**	61-64
5.1	System Analysis	61
5.2	Testing Methodology and Results	62
5.3	Related Work done	63
5.4	Comparison with Existing Systems	64
6	**Chapter 6 Conclusion and Future work**	65-69
6.1	Conclusion	65
6.2	Future work	66
6.3	References	67

List of Figures

Sr. No.	Name of the Figure	Page No.
3.1	Gantt Chart (Timeline)	14
3.3	Use Case Diagram	18
3.4	Entity-Relationship Diagram (ERD)	20
3.5	System Flow Diagram	22
3.6	Class Diagram	27
3.7	Sequence Diagram	28
3.8	State Diagram	30
3.9	Menu Tree Architecture	31
4.1	Authentication and Security Interface	51-52
4.2	The Main Dashboard & Document Upload Zone	53
4.3	The AI Hiring Insights Report	55
4.4	The Job Matcher Matrix	57
4.5	The Placement Prep Hub (Aptitude Module)	58
4.6	The User History Interface	60

List of Table

Sr. No.	Name of the Table	Page No.
3.2	Event Table	17
4.1.1	Users Collection	33
4.1.2	Resumes Collection	33
4.1.3	AI Insight	34
5.4	Comparison Table	63

Abstract

Today's job market is almost entirely controlled by automated Applicant Tracking Systems (ATS). This creates a massive problem for students because they get rejected by machines without ever being told why. On top of that, if a student wants to use an AI tool to check their resume, they are usually forced to upload their private information to cloud servers, which is a huge security risk. Even worse, the whole process of getting ready for placements is scattered across dozens of different websites, leaving students confused about where to even start. To solve these exact problems, this project introduces CareerPulse AI, a privacy-focused career platform. Built entirely from scratch using the MERN stack (MongoDB, Express.js, React, Node.js), this web app acts as a smart diagnostic tool that gives students instant, practical advice on how to fix their resumes.

The biggest technical feature of this project is how it handles data privacy. Instead of sending user resumes out to the internet, CareerPulse AI runs a local Large Language Model (Ollama gemma3:1b) directly on the backend server. This guarantees that user data never leaves the system. When a document is uploaded, the code first runs a strict check to ensure it is a valid professional resume. Then, instead of just spitting out a random score out of 100, the system uses Explainable AI to actually tell the user what they did wrong. The AI writes a custom technical summary and generates exactly eight interview questions based only on the specific projects the student mentioned. We also built a custom backend engine that mathematically compares the student's skills against real industry jobs, clearly pointing out what technologies they still need to learn. Users can even track their scores over time to see if their profile is actually improving with each new resume upload.

Fixing a resume is just the first step, so we also built a complete Placement Readiness Hub right into the dashboard. Here, students can take practice quizzes for coding, logical, verbal, and quantitative aptitude rounds without having to open another website. This system isn't just helpful for individual students, either. College placement cells can use the underlying data to see exactly which skills their entire graduating batch is struggling with, helping them plan better training workshops. In the end, CareerPulse AI connects what a student writes on paper with what the IT industry actually wants. By showing students exactly how automated hiring software works, the platform takes the fear and guesswork out of applying for jobs. It gives users the honest feedback and the practice tools they need to turn the stressful campus placement process into something secure, structured, and easy to manage.

Chapter 1

Introduction

This chapter provides the foundational context and strategic overview of the CareerPulse AI system. It details the transition of the modern recruitment landscape towards automated Applicant Tracking Systems (ATS) and identifies the critical transparency gap currently faced by students and fresh graduates. To establish the project's relevance and academic necessity, this chapter deeply explores the core motivations driving the development, defining the "Black Box" problem and the urgent need for privacy-centric AI solutions. It also explicitly outlines the project's technical scope, primary objectives, and the key stakeholders who will benefit from this comprehensive career preparation ecosystem.

1.1 Introduction

The way companies hire people today relies much more on software than it does on human review. Because every open job posting now receives hundreds or even thousands of applications over the internet, it is completely impossible for an HR team to sit down and read every single PDF. To handle this massive volume, corporations use Applicant Tracking Systems (ATS) as an automated frontline filter.

These enterprise systems work by scanning uploaded resumes and looking for specific keyword matches, formatting rules, and structural layouts. While this saves a massive amount of time for the company, it creates a deeply frustrating experience for the job seeker, especially for college students and fresh graduates. Candidates submit their resumes into a portal and simply receive an automated rejection email a week later. They are never told exactly why they failed. They are left guessing if they missed a specific keyword, if their PDF format was broken, or if they just lacked the right skills.

CareerPulse AI was built entirely on the MERN stack (MongoDB, Express.js, React.js, Node.js) to fix this exact problem. It acts as a personal diagnostic tool that lets students see their professional profiles exactly how an ATS sees them. By writing custom code to extract text from PDFs and Word documents, and then feeding that text into a locally hosted Large Language Model (LLM), the platform does much more than just give a random score out of 100. It actively points out

missing skills, predicts which job roles fit best based on the detected tech stack, and provides an integrated dashboard for actual aptitude practice.

1.2 Description

CareerPulse AI is a multi-part web application designed to simulate how real corporate recruitment software evaluates a candidate. The process starts when a user securely uploads their resume file. Before the server wastes any processing power on a heavy AI task, the backend runs an "Ultra Strict Rejection Logic" algorithm. This validation step uses regular expressions (Regex) to scan the document and ensure it actually looks like a professional resume. It checks for standard section headers (like Education, Experience, and Skills) and verifies that there is valid contact information. If someone uploads a blank document or a random image, the system rejects it immediately.

Once the resume passes the initial security check, the Node.js backend runs several background tasks at the same time:

- **AI-Generated Hiring Reports:** Using a self-hosted Ollama (gemma3:1b) model, the backend reads the extracted text and generates a concise technical summary. Even more importantly, the AI creates exactly eight custom interview questions based solely on the specific projects and technologies the student actually listed. This helps the student practice defending the claims they made on paper.
- **Interactive Job Matching:** The system takes the array of skills found in the resume and runs a comparative mathematical analysis against twelve different industry role templates (like Full Stack Developer, Data Analyst, or DevOps Engineer). It shows the user exactly which skills they already have and which ones they are missing, creating a personalized roadmap for what they need to learn next.
- **Placement Readiness Hub:** Because having a good resume is only the first part of getting a job, the application includes a dedicated Aptitude module on the React frontend. Here, students can take practice quizzes for quantitative, logical, verbal, and coding rounds, which are usually the first elimination tests in campus placements.
- **Performance Tracking:** Finally, the system calculates an overall ATS compatibility score and suggests the most realistic job role for the user based on how frequently certain technical keywords appeared in their document.

1.3 Stakeholders

Identifying and understanding stakeholders is essential for establishing the diverse educational and professional requirements the project must fulfill. The key stakeholders for the CareerPulse AI ecosystem include:

- **Students and Fresh Graduates:** The primary beneficiaries and end-users of the platform. They utilize the dashboard to receive immediate, actionable, and private feedback on their resumes, identify their technical weaknesses, and practice for upcoming technical and HR interview rounds.
- **Campus Placement Cells & Universities:** Educational administrators and placement officers can utilize the underlying architecture of this platform as a diagnostic tool. It allows them to assess the overall industry-readiness of a graduating batch, identify common skill deficiencies across the student body, and tailor their training programs accordingly.
- **Technical Mentors and Career Coaches:** Industry professionals and academic mentors can leverage the AI-generated hiring reports to bypass the initial screening phase. By using the AI-suggested project-specific interview questions, mentors can provide highly personalized, targeted verbal mock interviews for students.
- **System Administrators & Developers:** Technical personnel responsible for deploying and maintaining the local LLM inference environment, ensuring MongoDB database security, optimizing backend parsing routes, and maintaining the overall MERN infrastructure.

1.4 Motivation

The driving motivation behind this project is rooted deeply in the need for student empowerment in an increasingly automated, opaque, and commoditized world of recruitment. The current challenges that inspired this work include:

- **The "Black Box" Barrier:** Currently, students have absolutely no visibility into why their resumes fail to pass corporate ATS filters. They submit applications into a void and receive generic rejection emails. This project is motivated by the desire to provide that missing transparency through Explainable AI (XAI), automated ATS scoring, and explicit gap analysis.
- **Privacy-Centric AI Operations:** A major concern with modern AI career tools is that they are entirely cloud-based. They force users to upload highly sensitive personal data (phone

numbers, addresses, employment history) to third-party corporate servers, often to be used as training data. By implementing a local, self-hosted instance of Ollama (gemma3:1b), this project proves that students can receive enterprise-grade AI insights on their own machines without ever compromising their data sovereignty.

- **Holistic Preparation Ecosystem:** There is a significant fragmentation in the current market. Students use one website to build a resume, another to check its ATS score, and completely different platforms to practice aptitude questions or coding. CareerPulse AI is motivated by the need to consolidate this fragmented journey into a unified, single "Placement Hub".

1.5 Scope of Project Work

The technical and functional scope of this project involves the complete architectural design, engineering, and quality testing of the CareerPulse AI ecosystem. It specifically covers:

- **Full-Stack Engineering:** Developing a highly responsive, modern React frontend interface and a scalable, asynchronous Node.js/Express backend, all supported by a flexible MongoDB database schema.
- **Document Intelligence & Parsing:** Developing robust, fault-tolerant backend services capable of ingesting complex PDF and DOCX layouts and extracting them into structured, normalized technical data arrays.
- **Asynchronous AI Processing:** Implementing intelligent background worker logic that allows the user interface to stay responsive and deliver immediate baseline results while the computationally heavy local LLM processes deep-dive technical reports in the background.
- **Educational Support Systems:** Architecting the frontend "Aptitude" and "Interview" modules to ensure students are actively prepared for the technical, quantitative, and logical phases of modern hiring, effectively closing the loop from resume creation to interview success.

Summary

This chapter focuses on the radical shift toward automated Applicant Tracking Systems (ATS) and the resulting "Black Box" transparency gap that heavily disadvantages job seekers. It introduces CareerPulse AI as a privacy-centric, MERN-based diagnostic tool designed to specifically solve this issue. The chapter outlines the system's core capabilities, including local LLM-driven resume analysis, algorithmic job matching, and an integrated aptitude preparation hub. It clearly defines the problem statement regarding severe data privacy risks from cloud APIs and the currently fragmented nature of career preparation tools. Furthermore, it details how the system validates document structure before processing to save computational resources. Finally, it identifies key stakeholders, illustrating how both individual students and university placement cells benefit from this unified, explainable ecosystem.

Chapter 2

Literature Survey

This chapter presents a rigorous literature survey and comparative analysis of existing methodologies within the automated recruitment domain. It critically evaluates the current landscape of enterprise Applicant Tracking Systems (ATS) and cloud-based AI analyzers to identify their technical, functional, and privacy-related limitations. To establish a clear developmental trajectory, this chapter defines the specific technological objectives of the CareerPulse AI project designed to overcome these market gaps. It also includes a comprehensive project timeline utilizing a Gantt chart to illustrate the Agile development phases from conception to final optimization.

2.1 Description of Existing System

The current landscape of recruitment technology is largely monopolized by enterprise-level Applicant Tracking Systems (ATS) and specialized web-based platforms designed exclusively to handle high-volume candidate filtering for HR departments. According to standard industry practices, these systems primarily utilize keyword density matching, boolean searches, and structural pattern recognition to categorize and rank applicants. Prominent existing solutions and methodologies include:

- **WebMD-style Structured Checkers:** Many early-stage career platforms and government portals rely on rigid, form-based inputs. In these systems, users must manually enter their data piece-by-piece into specific database fields rather than uploading a free-form document.
- **Keyword-Centric ATS:** Traditional corporate systems (such as Workday, Taleo, or Greenhouse) operate as advanced search engines for recruiters. They extract text from a submitted PDF and run it against a strict Boolean string of mandatory skills. If a candidate possesses the skill but uses a slightly different synonym, the system often discards their application entirely.
- **Cloud-Based AI Analyzers:** The newest generation of consumer-facing tools utilizes third-party Large Language Models (LLMs) via paid APIs. While these provide excellent

natural language summaries, they mandate constant internet connectivity, incur high per-query costs, and require the transmission of personal data to external servers.

- **Manual Evaluation:** In many small-to-medium enterprises (SMEs), the "system" remains a manual review process. Eye-tracking studies reveal that a human recruiter scans a PDF for an average of just six to ten seconds, hunting solely for recognizable project titles.

2.2 Limitations of Present System

Despite rapid advancements in HR automation, the literature reveals several critical, systemic limitations in current recruitment technologies that actively disadvantage job seekers:

- **Complete Lack of Explainability:** Most existing ATS platforms operate as "Black Boxes." They provide a binary "Pass/Fail" result without offering any qualitative explanation as to *why* a candidate was rejected. This lack of transparency prevents candidates from learning and iterating on their profiles.
- **Privacy and Security Risks:** Centralized, cloud-based AI systems require the direct upload of highly sensitive personal data—making users highly vulnerable to data breaches and unauthorized corporate data harvesting.
- **Rigid Input Requirements and Parsing Failures:** Many legacy systems utilize outdated Optical Character Recognition (OCR) or basic text extraction that struggles to read non-standard resume layouts, multi-column designs, or graphical elements.
- **High Dependency on Paid APIs:** The heavy reliance on commercial APIs makes advanced AI career tools financially inaccessible for the average college student. Furthermore, it introduces a critical point of failure.
- **Absence of Integrated Preparation:** Existing platforms treat resume analysis and interview preparation as two completely separate industries, resulting in a fragmented user experience.

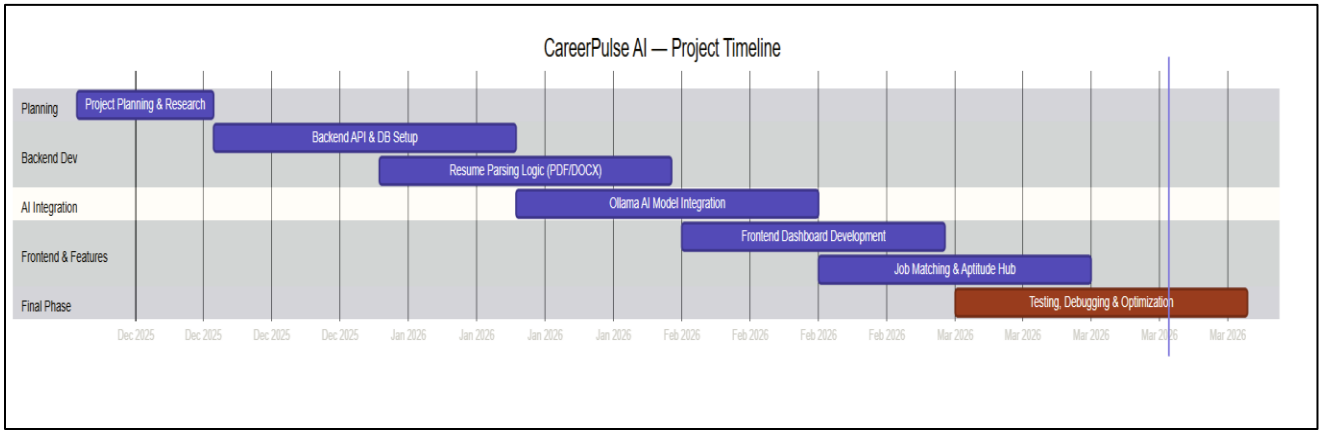
2.3 Objectives of the Project Work

Based on the explicit technological and functional gaps identified in the literature survey, the specific, measurable objectives of the CareerPulse AI project are:

- **To implement a locally-hosted LLM (Ollama gemma3:1b):** To engineer a system that executes complex generative AI tasks directly on the host machine, thereby ensuring 100% data privacy and zero cloud dependency for student resumes.
- **To develop an automated Job Matching algorithm:** To move beyond simple scoring by creating a logic system that dynamically cross-references extracted skills against specific industry roles, explicitly identifying which technologies a student is missing to become highly employable.
- **To provide Explainable AI (XAI) feedback:** To dismantle the "Black Box" by generating personalized, project-specific interview questions. This transforms the analysis from a passive score into an active validation of the student's actual claimed experience.
- **To integrate a unified Placement Prep Hub:** To centralize the job-seeking process by offering an interactive, real-time practice environment for the technical, quantitative, and logical rounds of the campus hiring process.

2.4 Gantt Chart (Timeline)

The project was developed over a structured period of four months. An Agile methodology was adopted to allow for iterative improvements, continuous testing, and the flexible integration of the local AI models as development progressed.



Summary

This section critically evaluates the current landscape of enterprise ATS and cloud-based AI recruitment tools, categorizing them into keyword-centric filters and rigid structured checkers. It highlights major systemic limitations, such as a complete lack of explainability, severe data privacy risks from external servers, and the financial barrier of paid AI services. The survey explicitly contrasts these flaws with the project's core objectives, establishing the absolute need for a locally hosted LLM (Ollama gemma3:1b) to ensure data sovereignty. It explains how traditional systems use rigid Boolean searches that unfairly discard qualified candidates. The chapter concludes by defining the developmental roadmap, utilizing an Agile methodology to achieve Explainable AI (XAI) feedback and dynamic job role alignment.

Chapter 3

Methodology and System Design

This chapter outlines the comprehensive methodology and architectural design framework underlying the CareerPulse AI platform. It details the strategic selection of the MERN stack and localized LLM inference engines to ensure absolute data sovereignty and asynchronous processing efficiency. To demonstrate the robust software engineering principles applied, this chapter thoroughly explores the system's logical data flows through multi-level Data Flow Diagrams (DFDs), Entity-Relationship Diagrams (ERDs), and comprehensive UML schemas. It establishes the exact structural blueprint required before transitioning into the practical coding and implementation phase.

3.1 Technologies Used and Their Description

The development of CareerPulse AI utilizes a modern, decentralized, full-stack architecture carefully selected to prioritize data sovereignty, high-speed asynchronous processing, and a seamless user experience.

- **MongoDB (Database Layer):** A highly scalable, document-oriented NoSQL database. It was chosen specifically for its ability to handle semi-structured data natively. MongoDB efficiently stores student profiles, raw extracted resume text, normalized skill arrays, and complex nested objects like the AI-generated JSON insights.
- **Express.js (Backend Framework):** A minimal, fast, and unopinionated Node.js web application framework. It provides a robust set of features to manage RESTful API endpoints and handles secure middleware routing.
- **React.jsx (Frontend Library):** An industry-standard, open-source JavaScript library utilized for building complex user interfaces. By utilizing React to construct a Single Page Application (SPA), the system can render the dynamic "Placement Prep Hub" without requiring cumbersome page reloads.

- **Node.js (Runtime Environment):** An open-source, cross-platform JavaScript runtime environment that executes backend code. Its event-driven, non-blocking I/O model makes it exceptionally well-suited for handling heavy-duty asynchronous operations.
- **Ollama - gemma3:1b (AI Engine):** A powerful, localized Large Language Model (LLM) framework that serves as the core analytical engine of the platform. By running the **gemma3:1b** model locally, the project successfully generates high-fidelity technical summaries without transmitting sensitive data to external cloud servers.
- **Tailwind CSS (Styling):** A utility-first CSS framework that allows for rapid UI development to build a highly professional, modern, and responsive interface.

3.2 Event Table

The event table provides a detailed, structured mapping of user-triggered actions and the corresponding programmatic system responses.

Table 3.2 :- Event Table

Event	Triggering Actor	System Action	Resulting Output
Resume Upload	Student	Multi-format parsing (PDF/DOCX) with ultra-strict validation	Validated Text Data
Deep-Dive Analysis	System	Ollama inference engine with prompt engineering	8–10 line summary & 8 questions
Role Matching	Student	Skill comparison with industry templates	Skill Gap % + Roadmap
Aptitude Assessment	Student	Randomized frontend questions	Score + Solutions

3.3 Use Case Diagram and Descriptions

The Use Case logic defines the functional boundaries of the system and illustrates exactly how the primary actor (the Student) interacts with various modules to achieve their career readiness objectives.

- **UC1: Profile Management:** Allows students to create secure accounts, authenticate via tokens, and maintain a persistent database history of their previously analyzed resumes for progress tracking.
- **UC2: Automated Resume Validation:** Before any heavy processing occurs, the system executes a series of strict regex and keyword checks to ensure the uploaded file is a genuine, professional resume (verifying contact info and section headers).
- **UC3: Technical Insight Generation:** The core intelligence of the system utilizes the local AI to read the student's project descriptions and generate highly tailored interview questions designed to challenge the student on their specific claims.
- **UC4: Career Path Alignment:** Students can view how their current skill set aligns with market demands. They select target roles, and the system calculates a percentage match score based on current industry requirements, explicitly listing missing technologies.

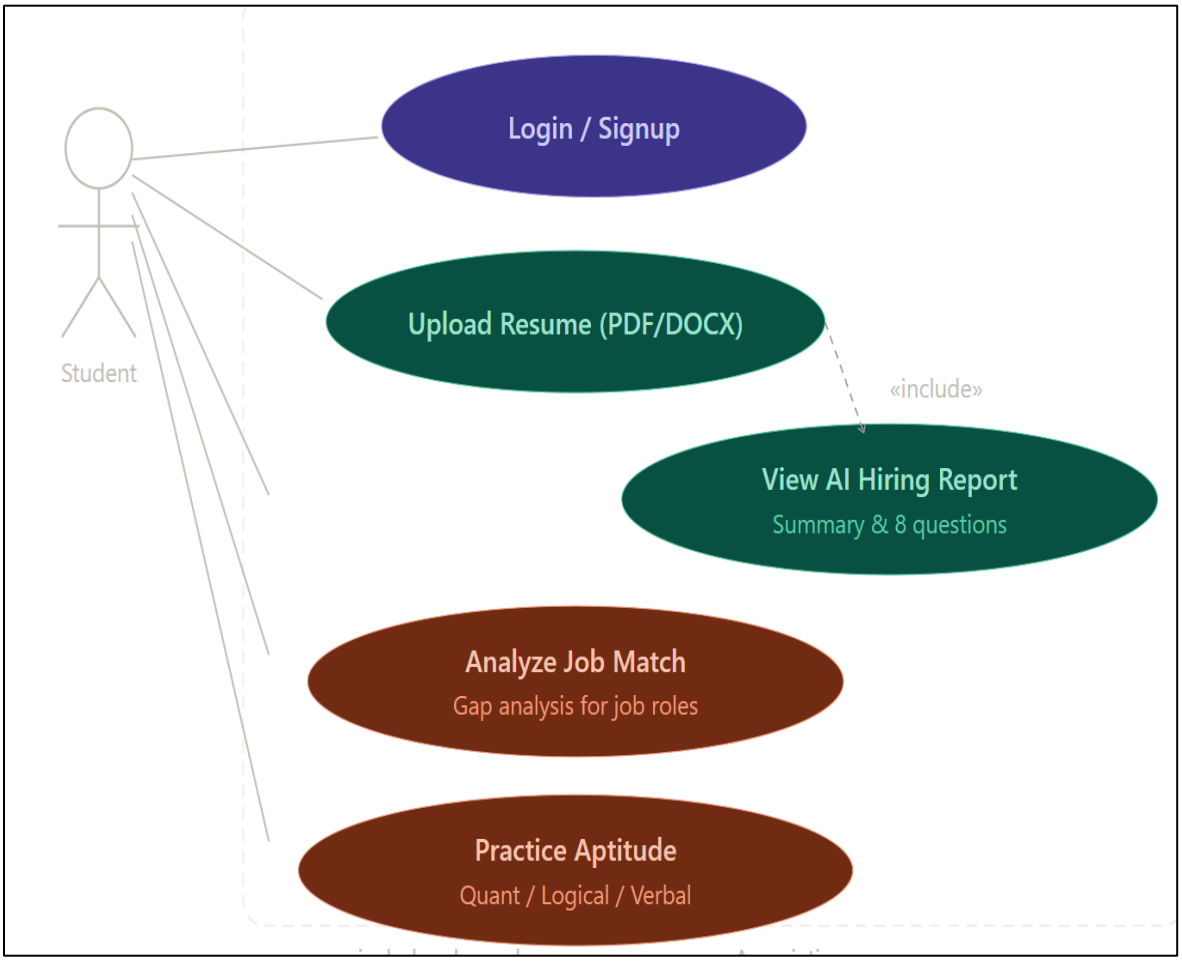


Fig.3.3 Use Case Diagram

3.4 Entity-Relationship Diagram (ERD)

The conceptual ERD illustrates the hierarchical data architecture within the MongoDB database, showing how various data points are interlinked to provide a cohesive user experience without data redundancy.

- **User Document:** The central, authenticated root document containing user credentials (hashed passwords, emails) and primary keys that link to the student's career history.

- **Resume Document:** A child document that stores the raw extracted text strings, file metadata (upload dates, original filenames), and computationally derived scores such as the ATS compatibility percentage and predicted roles.
- **Insights Sub-document:** An embedded, nested JSON structure within the Resume document containing the AI-generated hiring report. This includes the textual summary array, the list of curated questions, and the market suitability explanation.

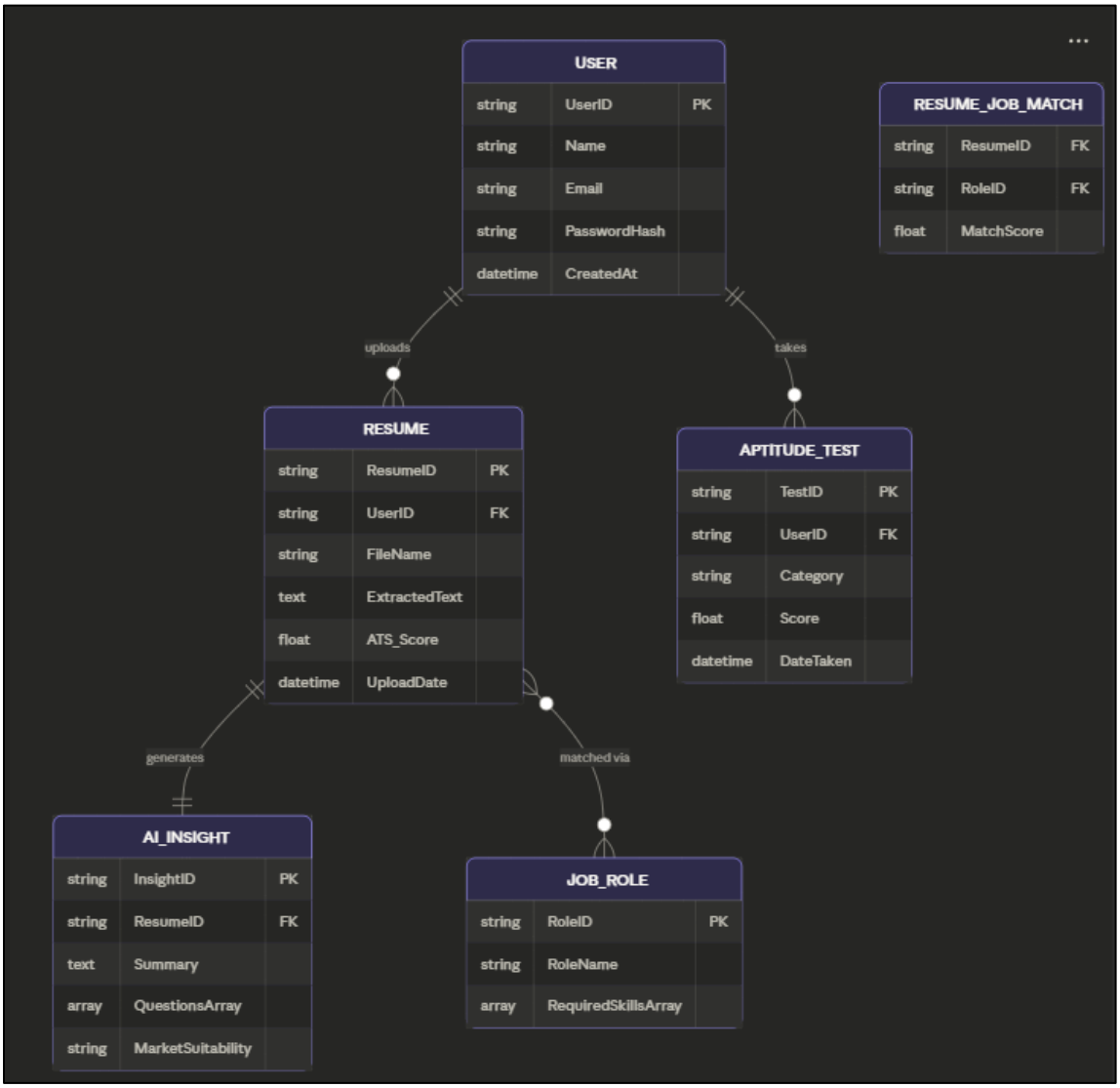


Fig.3.4 Entity Relationship Diagram

3.5 Flow Diagram

The system flow represents the logical, step-by-step data pipeline of the CareerPulse AI system, from initial file input to the final visualization of results.

1. **File Ingestion:** The student uploads a document (PDF or DOCX) through the secure React interface.
2. **Parsing Service:** The Express backend receives the buffer and utilizes libraries like pdf-parse or mammoth to convert the binary file data into searchable, raw text strings.
3. **Logical Filtering:** The system applies structural section detection to verify the presence of mandatory professional sections (Education, Skills, Experience) to prevent garbage data processing.
4. **Inference Layer:** The validated text block is packaged into a specific prompt and passed to the local Ollama service via HTTP POST for technical report generation.
5. **Data Persistence:** The AI returns a JSON object, which the backend sanitizes, structures, and saves to MongoDB before returning a success code to the frontend for display.

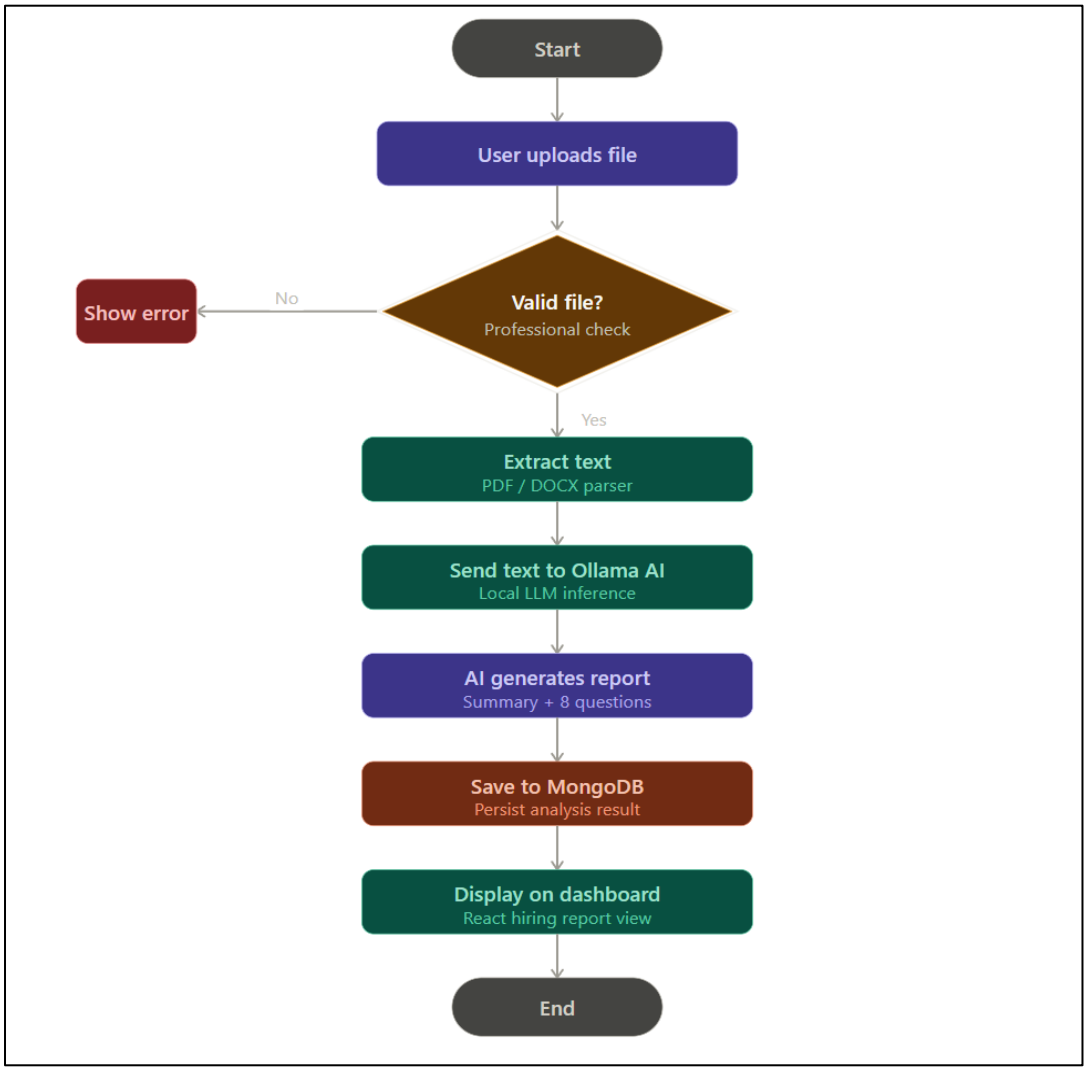
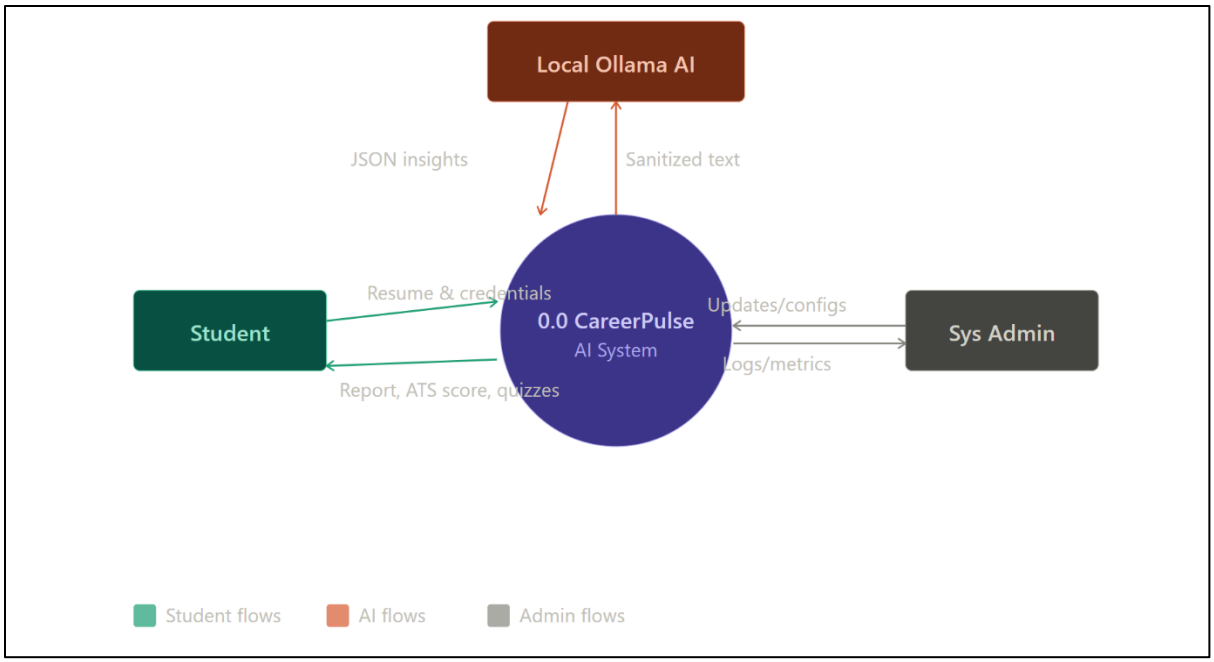


Fig.3.5 Flow Diagram

3.5.1 DFD Level 0 :-

The Context Diagram establishes the system's boundary. The primary entity is the **Student**, who inputs PDF/DOCX files and authentication data into the centralized CareerPulse System. The system processes this and returns visual Dashboards, Dashboards, and Quizzes. The system also

interacts with a secondary external entity, the **Local Ollama AI**, sending raw text and receiving structured JSON arrays back.

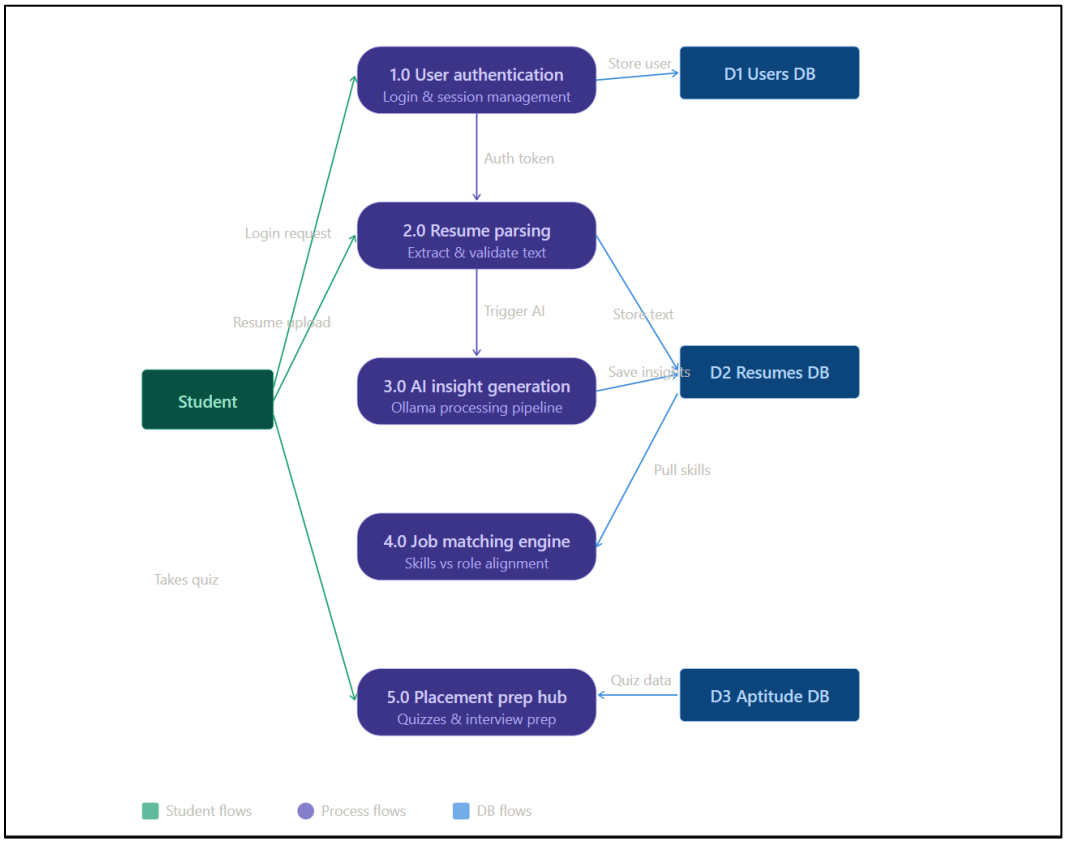


3.5.1 DFD Level 0 (Context Diagram)

3.5.2 DFD Level 1 (System Processes)

Level 1 breaks the system into five core functional sub-processes:

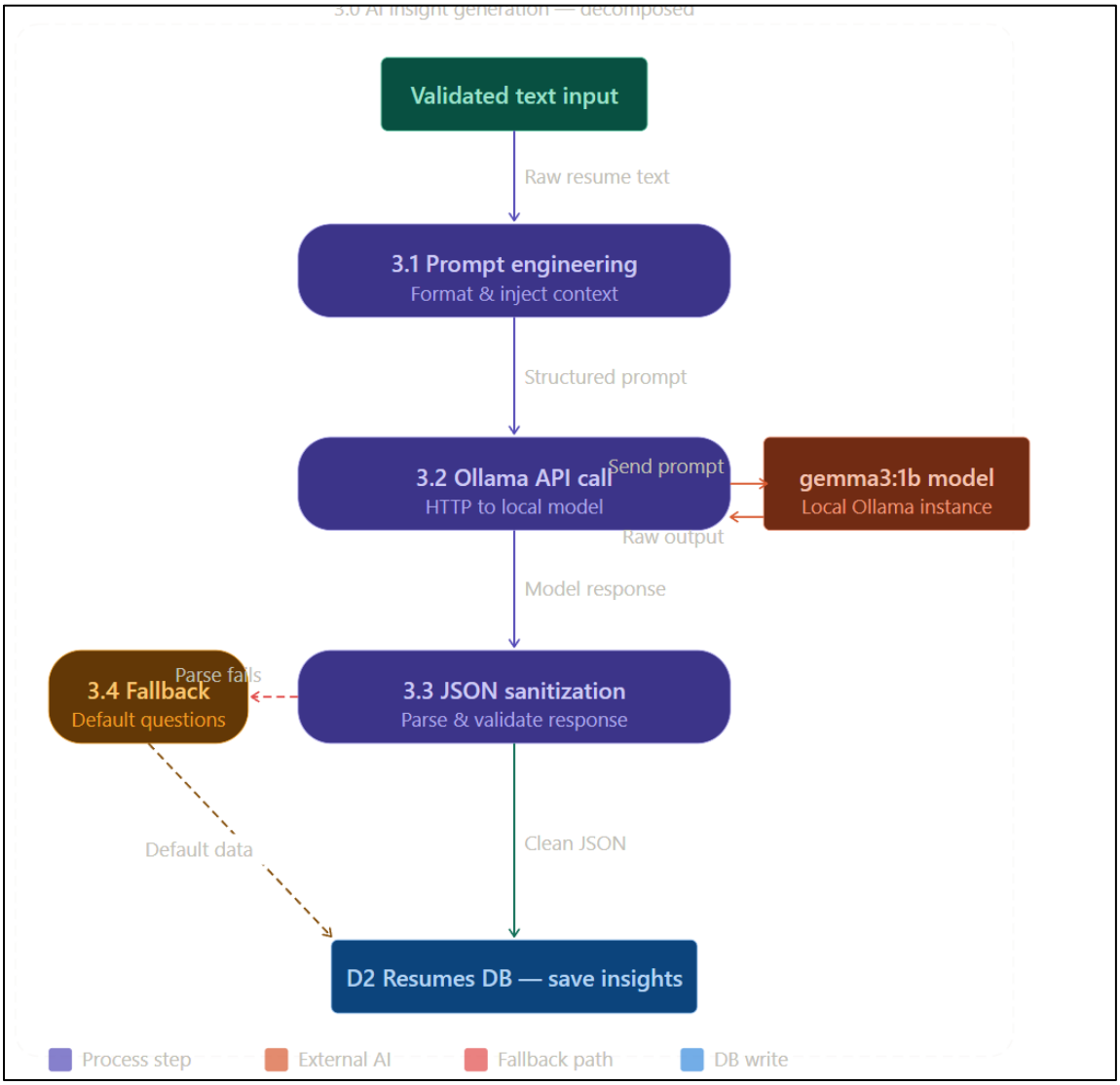
1. **User Authentication:** Validates identity and issues JWTs.
2. **Resume Parsing:** Extracts text and stores it in the Resumes Database.
3. **AI Insight Generation:** Communicates with the LLM to generate insights.
4. **Job Matching Engine:** Compares database skill arrays against stored role templates.
5. **Placement Prep Hub:** Fetches aptitude data for user practice.



3.5.2 DFD Level 1

3.5.3 DFD Level 2 (AI Insight Processing)

Zooming into Process 3.0 (AI Generation), the data flow undergoes complex handling. Raw text enters the **Prompt Engineering** phase. It is then sent to the **Ollama API**. The returning raw string enters the **JSON Sanitization** process (stripping backticks and escaped quotes). If the sanitization succeeds, data is stored. If it fails due to AI hallucination, the flow routes to the **Fallback Execution** node, which guarantees the user still receives 8 standard questions.



3.5.3 DFD Level 2

Zooming into Process 3.0 (AI Generation), the data flow undergoes complex handling. Raw text enters the **Prompt Engineering** phase. It is then sent to the **Ollama API**. The returning raw string enters the **JSON Sanitization** process (stripping backticks and escaped quotes). If the sanitization succeeds, data is stored. If it fails due to AI hallucination, the flow routes to the **Fallback Execution** node, which guarantees the user still receives 8 standard questions.

3.6 Class Diagram

The Class architecture provides a structural view of the backend's modular design. To ensure a highly scalable and maintainable codebase, the system implements a strict separation of concerns, dividing the architecture into Controllers, Middleware, Services, and Database Models.

- **Controllers Layer:** Acts as the entry point for API routes, managing HTTP request/response lifecycles.
    - **ResumeController Class:** Orchestrates file handling via Multer, validates inputs, and delegates the heavy computational lifting to the internal logic services.
    - **AuthController Class:** Manages secure user registration and login workflows, utilizing cryptographic hashing and returning JSON Web Tokens (JWT).
- **Middleware Layer:**
    - **AuthMiddleware Class:** Intercepts secure routes to verify incoming JWTs, ensuring only authenticated users can trigger the AI pipeline or access sensitive career data.
- **Services Layer:** Contains specialized business logic to keep the controllers lightweight.
    - **ResumeParser Class:** A dedicated utility responsible for the robust extraction of raw text strings from binary PDF and DOCX file formats.
    - **JobMatchService Class:** Encapsulates the deterministic logic for storing industry role templates, executing skill array intersections, and calculating percentage-based gap analyses.
    - **AIService Class:** Manages complex prompt construction, configures local model parameters (e.g., temperature settings), executes aggressive regex cleanup to parse JSON responses returned by the local LLM, and triggers static fallbacks in the event of AI hallucinations.
- **Database Models Layer (Mongoose):** Defines the strict NoSQL schemas and handles data persistence.
    - **User & Resume Classes:** Represent the primary database collections, maintaining a one-to-many relationship where a single user can track multiple resume iterations.

- **AI_Insight Class:** Implemented as an embedded document within the Resume class. This compositional relationship optimizes MongoDB read performance by preventing expensive, SQL-style JOIN queries when fetching hiring reports.

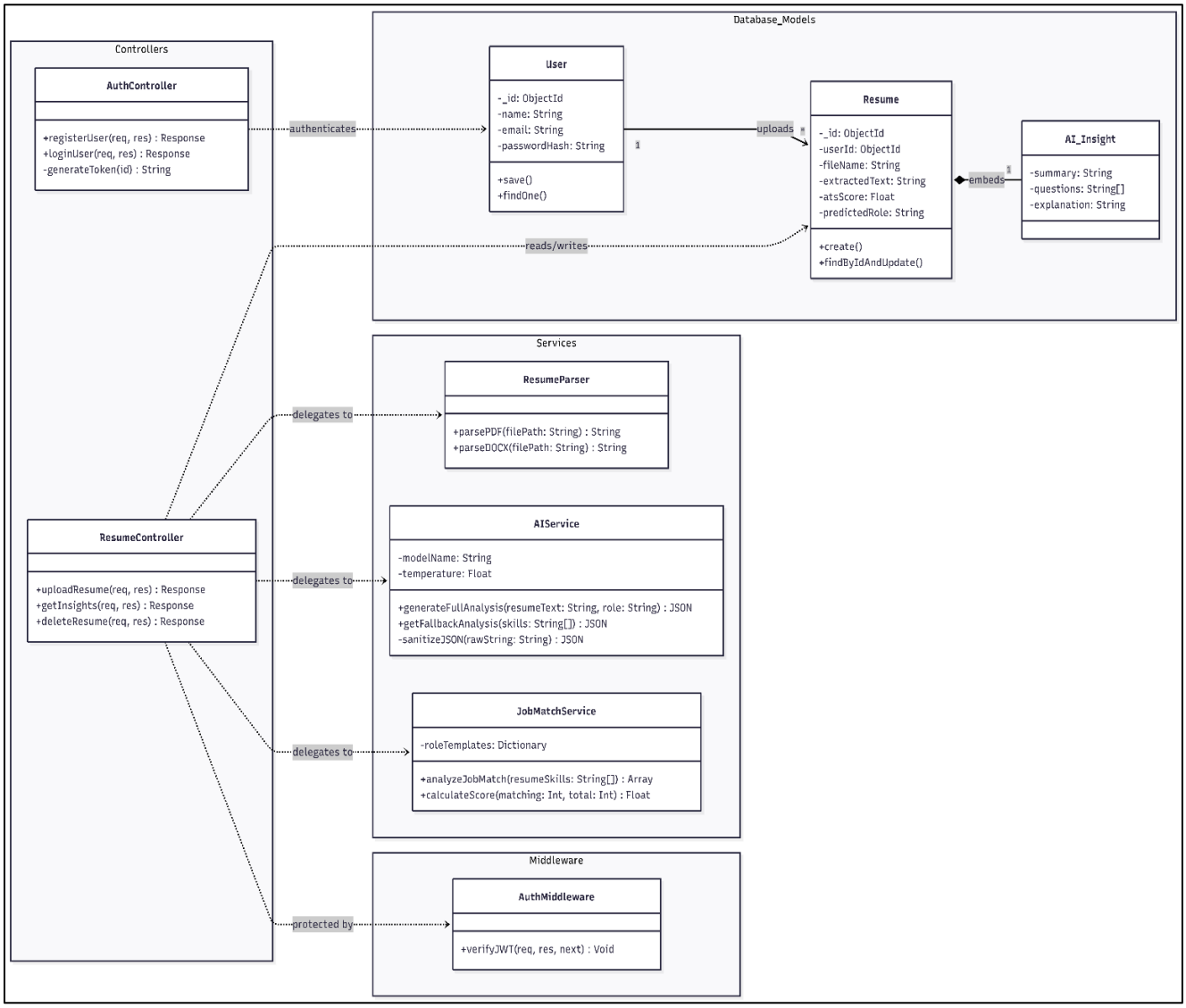


Fig.3.6 Class Diagram.

3.7 Sequence Diagram

This diagrammatic concept tracks the chronological flow of messages and API calls between the system components during a typical asynchronous analysis session.

1. **Student** initiates a file upload on the client interface.
2. **Frontend React app** packages the file into FormData and sends a POST request to the Express API.
3. **Express Backend** immediately performs local parsing and notifies the student of a successful upload (returning a 201 status), ensuring the UI doesn't freeze.
4. **Background Worker** (via a setTimeout delay) triggers the Ollama AI to process the heavy technical report invisibly in the background.
5. **Dashboard** polls the insights endpoint; once the AI completes the task and updates MongoDB, the UI displays the final report.

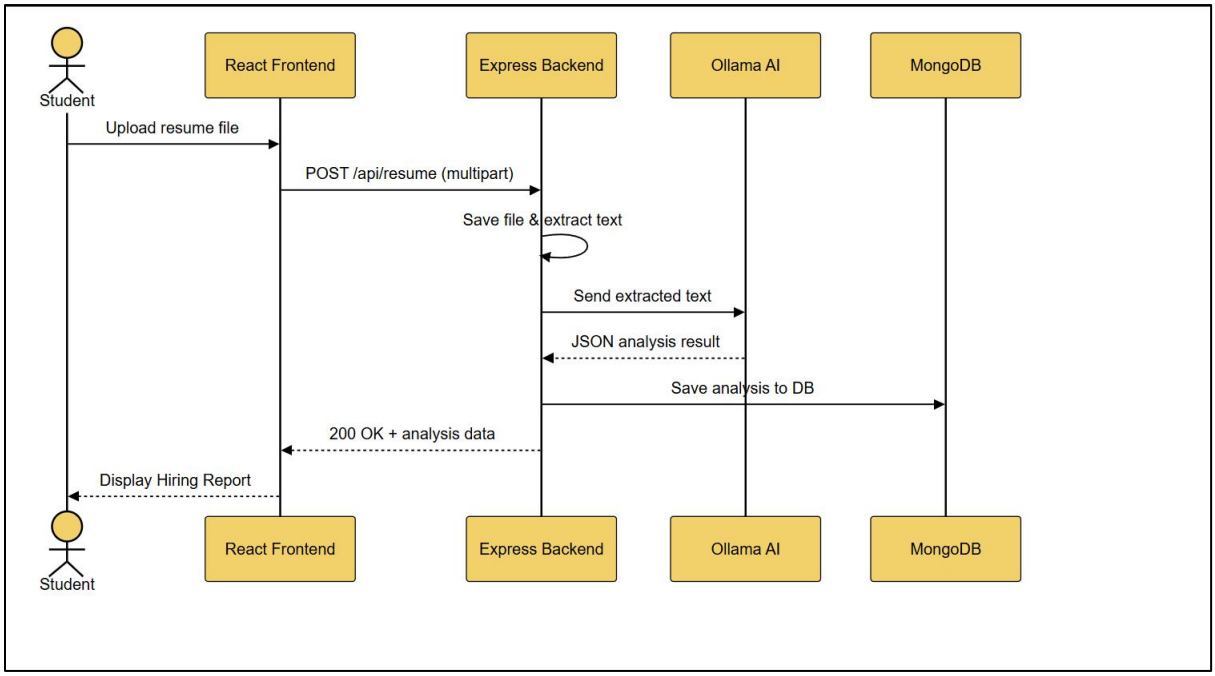


Fig.3.7 Sequence Diagram

3.8 State Diagram

The State Diagram illustrates the various operational phases and conditional transitions a resume undergoes within the system's lifecycle.

- **Initialized:** The user has logged in and is viewing the upload screen.
- **Parsing/Validating:** The system is actively extracting text and running rejection logic. (Transition to **Rejected** if the file is an image or lacks contact info).
- **Processing (AI-Running):** The valid text has been saved, and the system is waiting for the local AI model to finish generating insights.
- **Analyzed/Completed:** The final stable state where the student can view their technical summary, interview questions, and job roadmaps indefinitely.

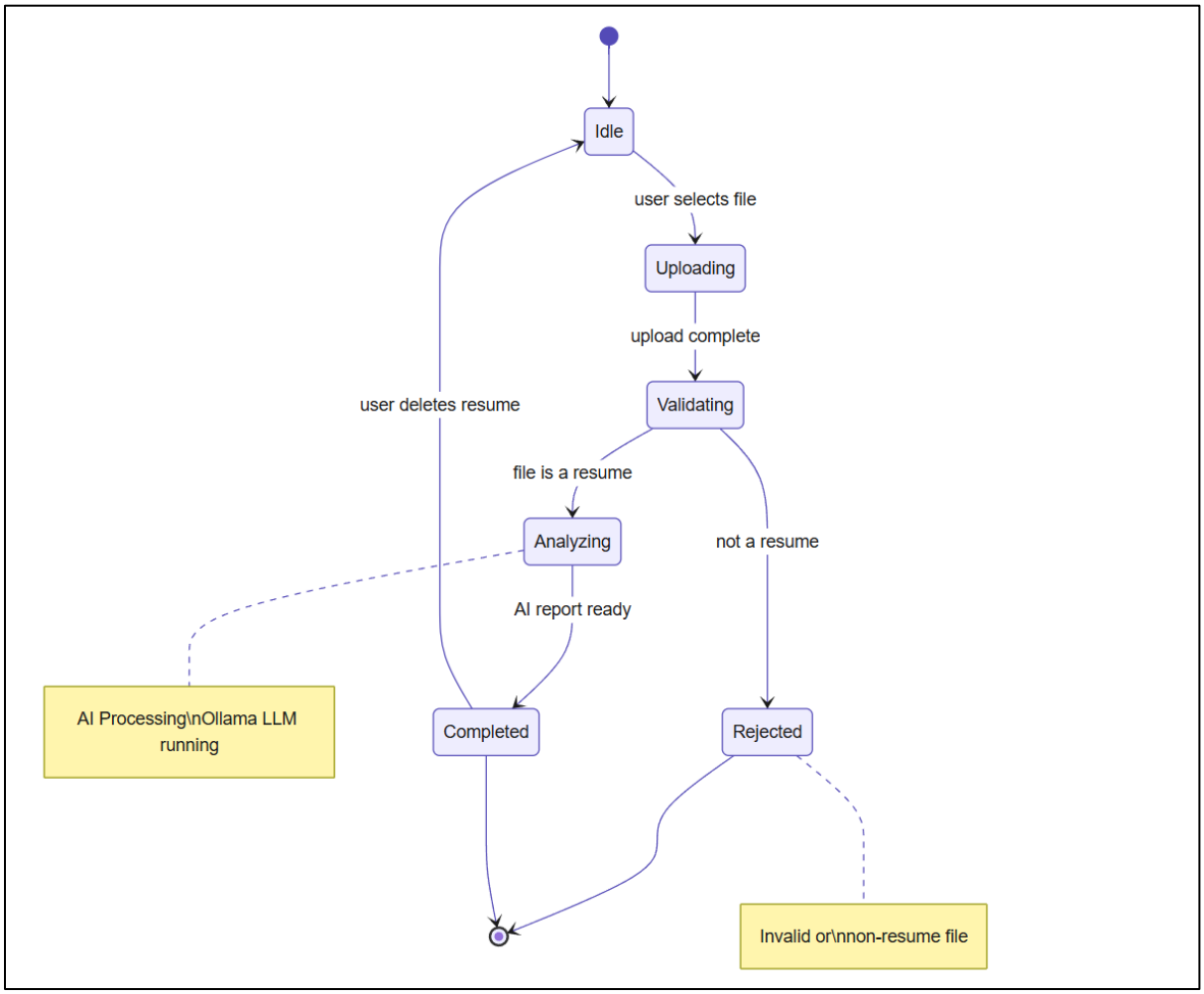


Fig.3.8 State Diagram

3.9 Menu Tree

The Menu Tree defines the site map, routing, and hierarchical navigation architecture of the React platform, ensuring an intuitive user journey.

- **Top-Level (Sidebar Navigation):** Includes persistent links to the Dashboard, Resume Analyzer, AI Insights, Job Matcher, and Placement Prep Hub.
- **Sub-Level (Analyzer):** Contains the drag-and-drop upload zone and a historical list of previously analyzed documents for comparison.

- **Sub-Level (Prep Hub):** Visually and functionally divided into distinct practice categories: Quantitative Aptitude, Logical Reasoning, Verbal Ability, and Pseudo-coding.

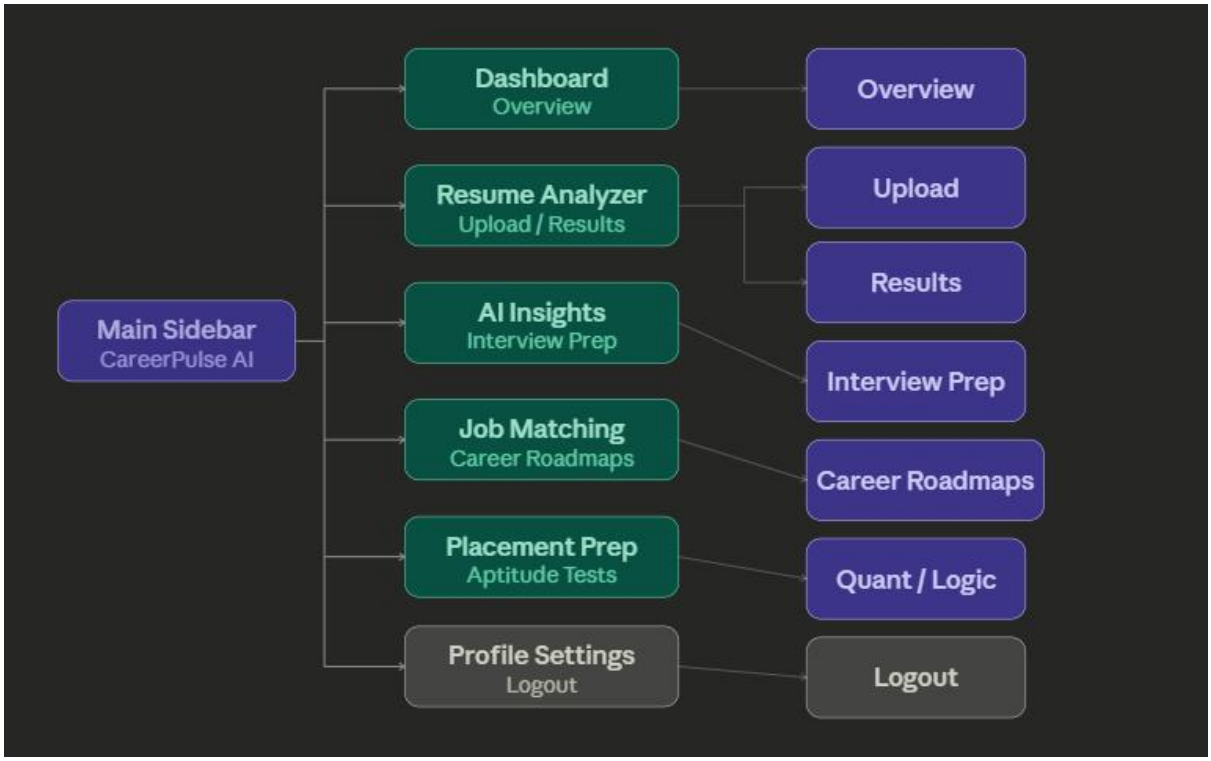


Fig.3.9 Menu Tree

Summary

This section critically evaluates the current landscape of enterprise ATS and cloud-based AI recruitment tools, categorizing them into keyword-centric filters and rigid structured checkers. It highlights major systemic limitations, such as a complete lack of explainability, severe data privacy risks from external servers, and the financial barrier of paid AI services. The survey explicitly contrasts these flaws with the project's core objectives, establishing the absolute need for a locally hosted LLM (Ollama gemma3:1b) to ensure data sovereignty. It explains how traditional systems use rigid Boolean searches that unfairly discard qualified candidates. The chapter concludes by defining the developmental roadmap, utilizing an Agile methodology to achieve Explainable AI (XAI) feedback and dynamic job role alignment.

Chapter 4

Implementation

This chapter provides the comprehensive technical implementation of the **CareerPulse AI** system. It details the transition from the architectural design phase into a fully functional, coded platform. To ensure academic rigor and demonstrate the extensive custom engineering involved beyond standard API integration, this chapter deeply explores the custom algorithmic coding for resume validation, static job-matching engines, and fail-safe sanitization logic. It also includes the exact database schemas with their constraints and detailed descriptions of the graphical user interfaces.

The implementation phase of the CareerPulse AI ecosystem involved translating the conceptual UML diagrams and logical data flows into a robust, production-ready MERN stack application. A critical focus during this phase was developing proprietary, static algorithmic logic to govern the system, ensuring that the platform is not merely a wrapper for an AI model, but a highly engineered, self-sustaining diagnostic tool.

4.1 List of Tables with Attributes and Constraints

This collection stores the primary authentication and identity details of the student.

Table 4.1.1: Users Collection

Attribute Name	Data Type	Constraints & Description
_id	ObjectId	Primary Key, auto-generated by MongoDB
name	String	Required. Stores the full name of the user
email	String	Required, unique, lowercase. Regex validated for standard email format
password	String	Required. Stored as a bcrypt hash for cryptographic security
createdAt	Date	Default: Date.now(). Timestamp of account creation

Table 4.1.2: Resumes Collection

This collection acts as the central repository for the analytical data extracted from the uploaded documents.

Attribute Name	Data Type	Constraints & Description
_id	ObjectId	Primary Key, auto-generated
userId	ObjectId	Foreign key referencing the Users collection
fileName	String	Required. Original name of uploaded PDF/DOCX file
extractedText	String	Required. Raw parsed text from the document
skillsDetected	Array<String>	List of normalized technical skills extracted via custom logic
atsScore	Number	Integer (0–100). Default: 0
predictedRole	String	Best-matched role based on skill frequency engine

Table 4.1.3: AI_Insights (Embedded Sub-document)

To optimize read performance and avoid complex SQL-style JOIN operations, the hiring report is stored directly inside the Resume document as an embedded object.

Attribute Name	Data Type	Constraints & Description
summary	`String`	Required. 9–10 line technical journey generated by the inference engine
questions	`Array<String>`	Array containing exactly 8 project-specific questions
explanation	`String`	Market suitability explanation

4.2 System Coding

The core functionality of the project is driven by highly specialized backend services. A significant portion of the development effort was dedicated to writing custom, static logic to evaluate candidates, structure data, and safeguard the AI engine from processing invalid inputs. The codebase is organized into five distinct architectural phases, moving from the foundational database layer up to the client interface.

Phase 1: Foundation (Database)

This phase establishes the connection to the data layer and defines the strict schemas required for NoSQL data persistence.

4.2.1 Database Configuration (Mongoose)

Purpose*:* Establishes a secure, asynchronous connection to the MongoDB cluster, handling potential connection timeouts or URI errors to prevent backend crashes.

config/db.js

```
const mongoose = require('mongoose');


const connectDB = async () => {
```

```
  try {
    const conn = await mongoose.connect(process.env.MONGODB_URI, {
      useNewUrlParser: true,
      useUnifiedTopology: true,
    });
    console.log(`MongoDB Connected Successfully: ${conn.connection.host}`);
  } catch (error) {
    console.error(`MongoDB Connection Error: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

4.2.2 Database Architecture (Mongoose Schemas)

Purpose*:* Defines the NoSQL schema structure for storing nested AI reports and tracking ATS scores securely without introducing relational bottlenecks.

models/resume.model.js

```
const mongoose = require('mongoose');
const resumeSchema = new mongoose.Schema({
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
```

```
  },
  fileName: { type: String, required: true },
  extractedText: { type: String, required: true },
  skillsDetected: [{ type: String }],
  atsScore: { type: Number, default: 0 },
  predictedRole: { type: String },

  // Embedded JSON Object for AI Insights to prevent expensive SQL-style JOINs
  aiInsights: {
    summary: { type: String },
    questions: [{ type: String }],
    explanation: { type: String }
  }
}, { timestamps: true });

module.exports = mongoose.model('Resume', resumeSchema);
```

Phase 2: Security & Middleware (The Gatekeepers)

This phase protects the system's endpoints, ensuring that only authenticated users can access the AI services and that uploaded files are safe to process.

4.2.3 Security & Authentication Middleware

Purpose: Intercepts HTTP requests to ensure only users with valid JSON Web Tokens (JWT) can access the secure backend services.

middleware/auth.middleware.js

```
const jwt = require('jsonwebtoken');

const verifyJWT = (req, res, next) => {

  try {

    const token = req.header("Authorization")?.replace("Bearer ", "");


    if (!token) {

      return res.status(401).json({ message: "Unauthorized request. Token missing." });

    }


    const decodedToken = jwt.verify(token, process.env.JWT_SECRET);

    req.user = decodedToken; // Attach verified user ID to the request lifecycle


    next(); // Pass control to the specific route controller

  } catch (error) {

    return res.status(401).json({ message: "Invalid or expired Access Token." });

  }

};


module.exports = verifyJWT;
```

4.2.4 File Upload Middleware (Multer)

Purpose*:* Intercepts incoming multipart/form-data requests, securely saves the binary document to local disk storage, and enforces strict MIME-type filtering for PDF and DOCX formats.

middleware/upload.middleware.js

```
const multer = require('multer');

const path = require('path');


// Configure local storage destination and unique filenames

const storage = multer.diskStorage({

  destination: (req, file, cb) => {

    cb(null, 'uploads/'); // Temporary storage for parsing

  },

  filename: (req, file, cb) => {

    const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() * 1E9);

    cb(null, file.fieldname + '-' + uniqueSuffix + path.extname(file.originalname));

  }

});


// Strict MIME type filtering to prevent malicious uploads

const fileFilter = (req, file, cb) => {

  const allowedMimeTypes = [

    'application/pdf',

    'application/vnd.openxmlformats-officedocument.wordprocessingml.document'

  ];
```

```
  if (allowedMimeTypes.includes(file.mimetype)) {
    cb(null, true);
  } else {
    cb(new Error('Invalid file type. Only PDF and DOCX documents are allowed.'), false);
  }
};

// Initialize Multer with a 5MB size restriction
const upload = multer({
  storage: storage,
  limits: { fileSize: 5 * 1024 * 1024 },
  fileFilter: fileFilter
});

module.exports = upload;
```

Phase 3: Core Business Logic (Services)

These specialized services handle the computationally heavy tasks, separating the business logic from the routing controllers.

4.2.5 Document Parsing Service

Purpose*:* A utility service that takes the binary file path saved by Multer and extracts the raw text strings required for AI processing using the pdf-parse library.

services/resumeParser.service.js

```
const fs = require('fs');
const pdfParse = require('pdf-parse');

const parseResume = async (filePath) => {
    try {
        const dataBuffer = fs.readFileSync(filePath);
        const data = await pdfParse(dataBuffer);

        let extractedText = data.text;
        extractedText = extractedText.replace(/\n+/g, ' ').trim();

        fs.unlinkSync(filePath); // Delete temp file to save disk space
        return extractedText;
    } catch (error) {
        console.error("PDF Parsing Error:", error.message);
        throw new Error("Failed to extract text from the document.");
    }
};

module.exports = { parseResume };
```

4.2.6 Local LLM Generation & Sanitization

Purpose: Communicates with the local Ollama (gemma3:1b) model via HTTP and aggressively sanitizes the output to ensure a valid JSON structure, effectively handling potential AI hallucinations.

JavaScript

```
const axios = require("axios");


const generateFullAnalysis = async (resumeText, role, skills) => {
  try {
    const response = await axios.post("http://localhost:11434/api/generate", {
      model: "gemma3:1b",
      prompt: `Generate JSON hiring report for: ${resumeText}`,
      stream: false,
      options: { temperature: 0.1 }
    });

    let raw = response.data.response;

    // Extract JSON block from potential AI conversational hallucination
    const firstBrace = raw.indexOf('{');
    const lastBrace = raw.lastIndexOf('}');
    let jsonText = raw.substring(firstBrace, lastBrace + 1);

    // Aggressive Regex Cleanup
```

```
      jsonText = jsonText
          .replace(/\\"/g, "")     // Fix escaped quotes
          .replace(/\\n/g, " ")     // Fix escaped newlines
          .replace(/,\s*]/g, "]")   // Strip trailing commas (arrays)
          .replace(/,\s*\}/g, "}"); // Strip trailing commas (objects)

      return JSON.parse(jsonText);

    } catch (error) {
      console.error("AI Generation Failed. Executing Static Fallback.");
      return getFallbackAnalysis(skills);
    }
};
```

4.2.7 Static Job Matching Engine

Purpose : A deterministic, mathematical algorithm that cross-references user skills against industry templates to calculate gap percentages.

JavaScript

```
const analyzeJobMatch = async (resumeSkills) => {
  const roleTemplates = {
    "Full Stack Developer": ["react", "node.js", "express", "mongodb", "javascript"],
    "Data Analyst": ["python", "sql", "excel", "tableau", "pandas"],
    "DevOps Engineer": ["docker", "kubernetes", "aws", "ci/cd", "linux"]
  };
```

```
const results = [];

for (const role in roleTemplates) {
    const roleSkills = roleTemplates[role];

    // Array Intersections
    const matchingSkills = roleSkills.filter(skill => resumeSkills.includes(skill));
    const missingSkills = roleSkills.filter(skill => !matchingSkills.includes(skill));

    const matchScore = Math.round((matchingSkills.length / roleSkills.length) * 100);

    if (matchScore >= 30) {
        results.push({
            role,
            matchScore,
            matchingSkills,
            missingSkills
        });
    }
}

return results.sort((a, b) => b.matchScore - a.matchScore);
```

```
};
```

Phase 4: The API Controllers (Execution)

These controllers handle the HTTP request lifecycles, tying the middleware and services together to execute the core features.

4.2.8 User Authentication Controller

Purpose : Handles secure user registration, cryptographic password hashing via bcryptjs, and login validation.

JavaScript

```
const User = require('../models/user.model');

const bcrypt = require('bcryptjs');

const jwt = require('jsonwebtoken');


const generateToken = (id) => {

  return jwt.sign({ id }, process.env.JWT_SECRET, { expiresIn: '30d' });

};


exports.registerUser = async (req, res) => {

  try {

      const { name, email, password } = req.body;


      const userExists = await User.findOne({ email });

      if (userExists) {

          return res.status(400).json({ message: "User already exists in the system." });
```

```
    }

    const salt = await bcrypt.genSalt(10);
    const hashedPassword = await bcrypt.hash(password, salt);

    const user = await User.create({
      name,
      email,
      password: hashedPassword
    });

    if (user) {
      res.status(201).json({
        _id: user.id,
        name: user.name,
        email: user.email,
        token: generateToken(user._id)
      });
    }
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};
```

4.2.9 Document Validation & Execution Controller

Purpose: The gateway controller that parses the uploaded file, executes strict rejection algorithms to save computational resources, and initiates the asynchronous AI background worker.

JavaScript

```
exports.uploadResume = async (req, res) => {
  try {
    const extractedText = await parseResume(req.file.path);
    const textLower = extractedText.toLowerCase();

    // 1. Strict Section Validation
    const sectionGroups = {
      education: ["education", "university", "college", "degree"],
      experience: ["experience", "work", "employment", "internship"],
      projects: ["projects", "portfolio", "github"],
      skills: ["skills", "technologies", "expertise"]
    };

    const foundSections = Object.keys(sectionGroups).filter(group =>
      sectionGroups[group].some(keyword => textLower.includes(keyword))
    );

    // 2. Regex Contact Validation
    const emailPattern = /[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/;
```

```
    const hasEmail = emailPattern.test(extractedText);

    if (foundSections.length < 3 || !hasEmail || extractedText.length < 600) {
        return res.status(400).json({ message: "Rejected: Invalid Resume Format." });
    }

    // 3. Save to DB & Trigger Asynchronous Background AI Worker
    const savedResume = await Resume.create({ /* ...document data... */ });
    res.status(201).json({ message: "Upload success. AI running in background." });

    setTimeout(async () => {
        const insights = await generateFullAnalysis(extractedText);
        await Resume.findByIdAndUpdate(savedResume._id, { aiInsights: insights });
    }, 1000);

  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};
```

Phase 5: The Client Side

This phase manages the frontend interactions, providing the user with an intuitive and responsive experience.

4.2.10 React Client State Management

Purpose: Manages the interactive state, randomized question rendering, and real-time score tracking for the Placement Prep Hub without mutating direct state objects.

JavaScript

```
import { useState, useEffect, useCallback } from "react";

// Data arrays imported from config files...


const AptitudePrep = () => {

  const [activeView, setActiveView] = useState("prep");

  const [sampleQuestions, setSampleQuestions] = useState([]);

  const [selectedAnswers, setSelectedAnswers] = useState({});


  // Memoized function to randomly shuffle and select 10 questions

  const generateRandomQuestions = useCallback(() => {

    const shuffled = [...questionPool].sort(() => 0.5 - Math.random());

    setSampleQuestions(shuffled.slice(0, 10));

    setSelectedAnswers({}); // Reset state on new batch

  }, []);


  useEffect(() => {

    generateRandomQuestions();

  }, [generateRandomQuestions]);


  // Handle user selection without mutating direct state
```

```
  const handleSelectOption = (questionId, option) => {
    setSelectedAnswers(prev => ({
      ...prev,
      [questionId]: option
    }));
  };

  /* UI Render Logic (JSX) Truncated.
    Handles conditional rendering between 'Study Guide' and 'Quiz Interface'
    and maps through sampleQuestions to generate interactive UI cards.
  */
};

export default AptitudePrep;
```

4.3 Screen Layouts and Report Layouts

The frontend of CareerPulse AI was developed using **React.jsx** and **Tailwind CSS**. The interface was designed to be highly intuitive, utilizing a dark-mode theme to reduce eye strain and provide a modern, professional corporate aesthetic.

4.3.1 Authentication and Security Interface

- **Layout Description:** The system utilizes a secure, JWT-backed authentication portal. The layout is designed with modern UI principles, utilizing high-contrast input fields for Name, Email, and Password. It features robust client-side validation to ensure user career data is securely isolated from the moment of account creation.

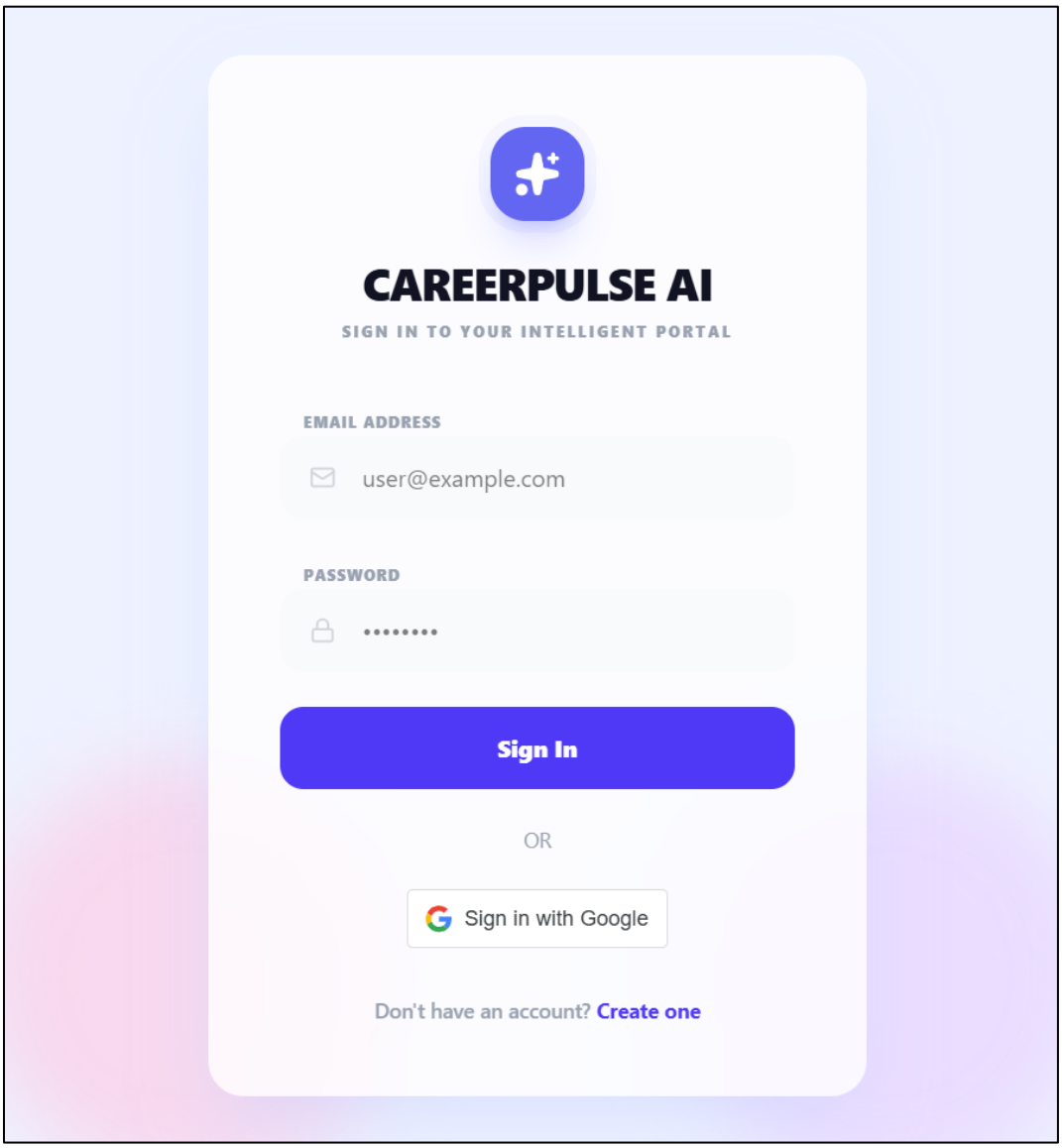


Fig.4.3.1:-Login page

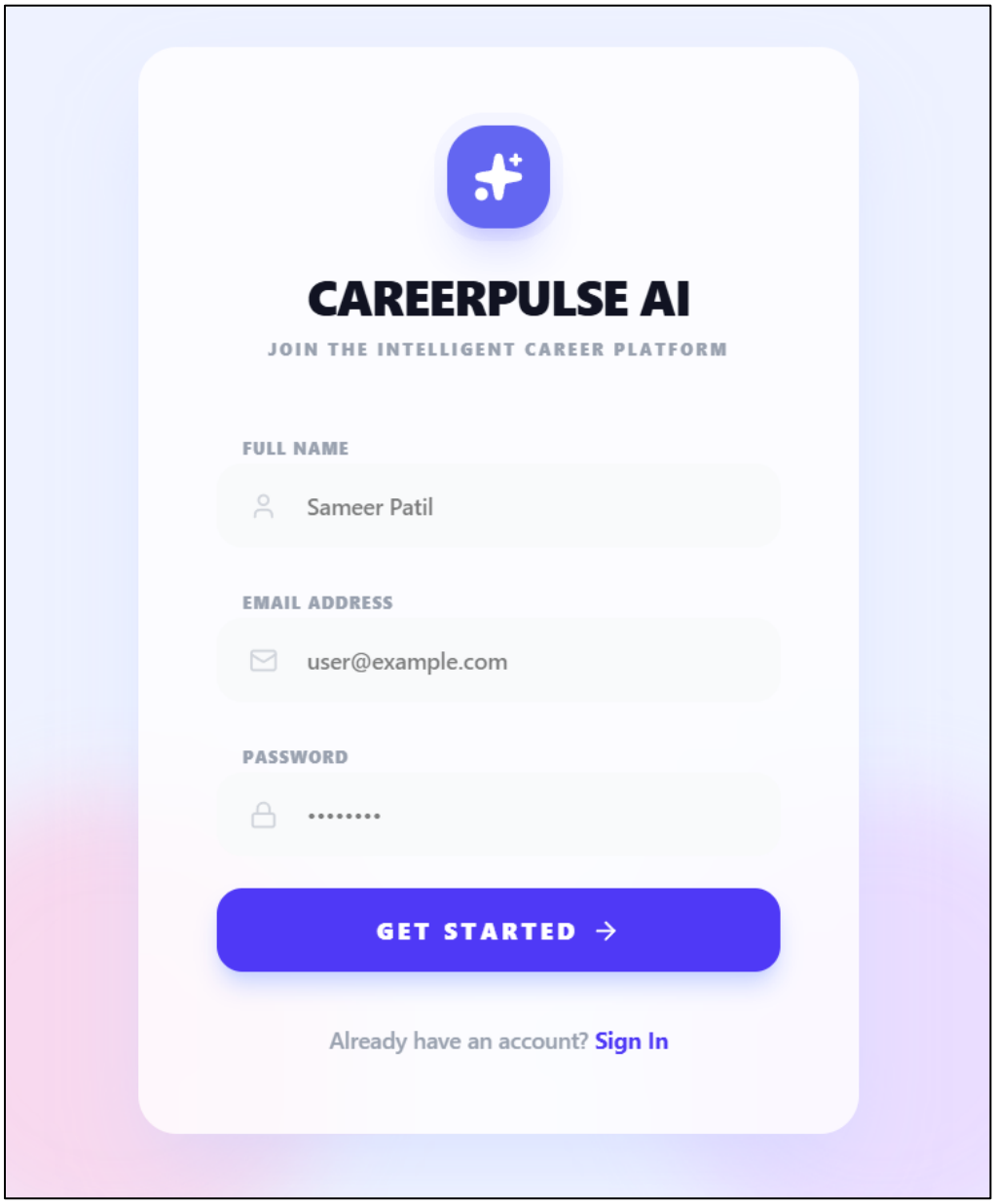


Fig.4.3.1 Sign-Up page

4.3.2 The Main Dashboard & Document Upload Zone

- **Layout Description:** Serving as the central interactive hub for the student, this layout features a prominent drag-and-drop zone optimized for capturing PDF and DOCX files. At the top of the interface, dynamic circular progress indicators (ProgressCircle.jsx) display the user's mathematically computed **ATS Compatibility Score** and a summary of their most recent upload history.
- **Interactivity:** Upon file selection, a smooth loading animation indicates that the custom backend parsing and background AI extraction processes have successfully initiated.

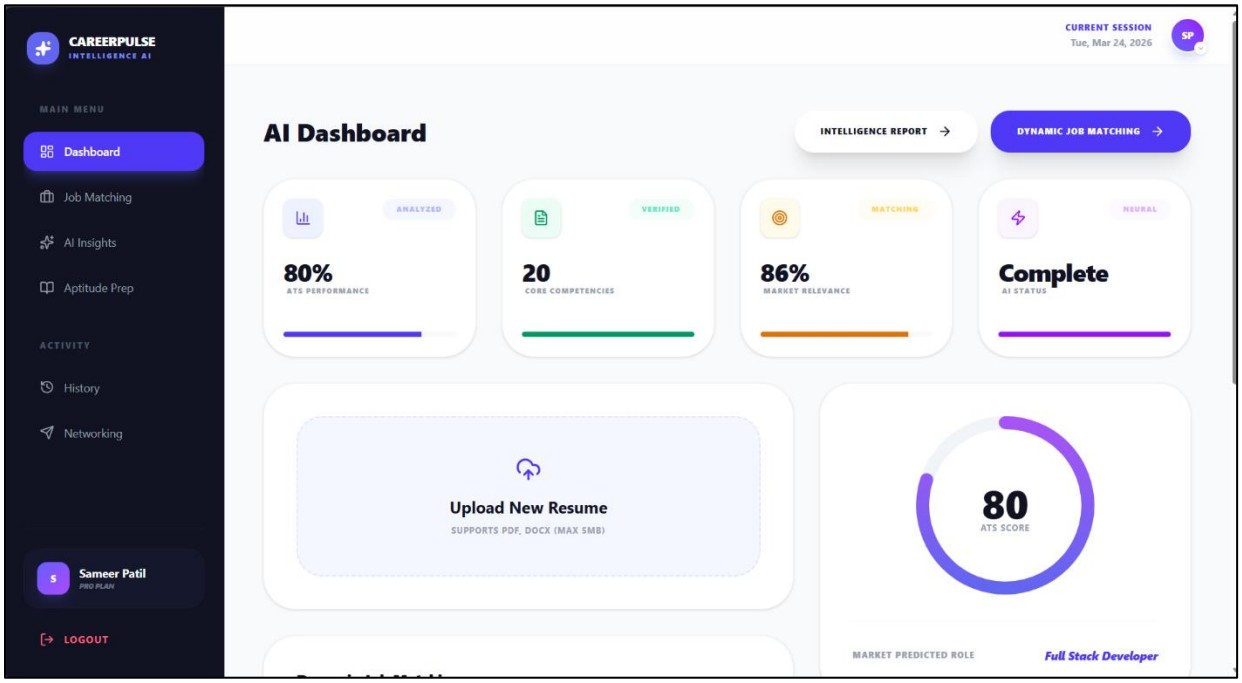


Fig.4.3.2:- Dashboard page

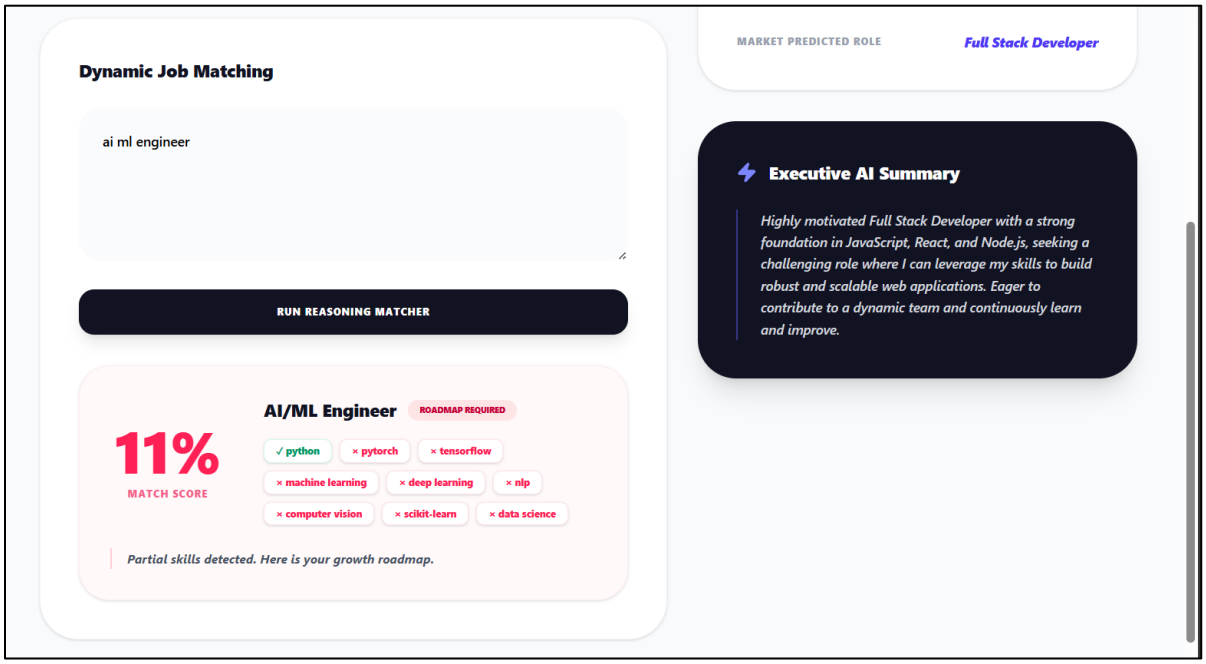


Fig.4.3.2:- Dashboard page

4.3.3 The AI Hiring Insights Report

- **Layout Description:** This specialized layout renders the deep-dive analysis generated by the local Ollama engine. It is horizontally segmented to maximize readability:
    1. **Technical Summary Card:** A clean, elevated text block displaying the 9-10 line professional narrative.
    2. **Interview Questions Accordion:** A dynamic list of the 8 customized technical questions. Each question is contained within a clickable dropdown (accordion style) that expands to reveal standard solutions and the green validation checkmarks.
- **Interactivity:** Users can click a "Refresh Set" button, which fires an asynchronous request to the backend to generate a brand new set of randomized questions based on the same static resume context.

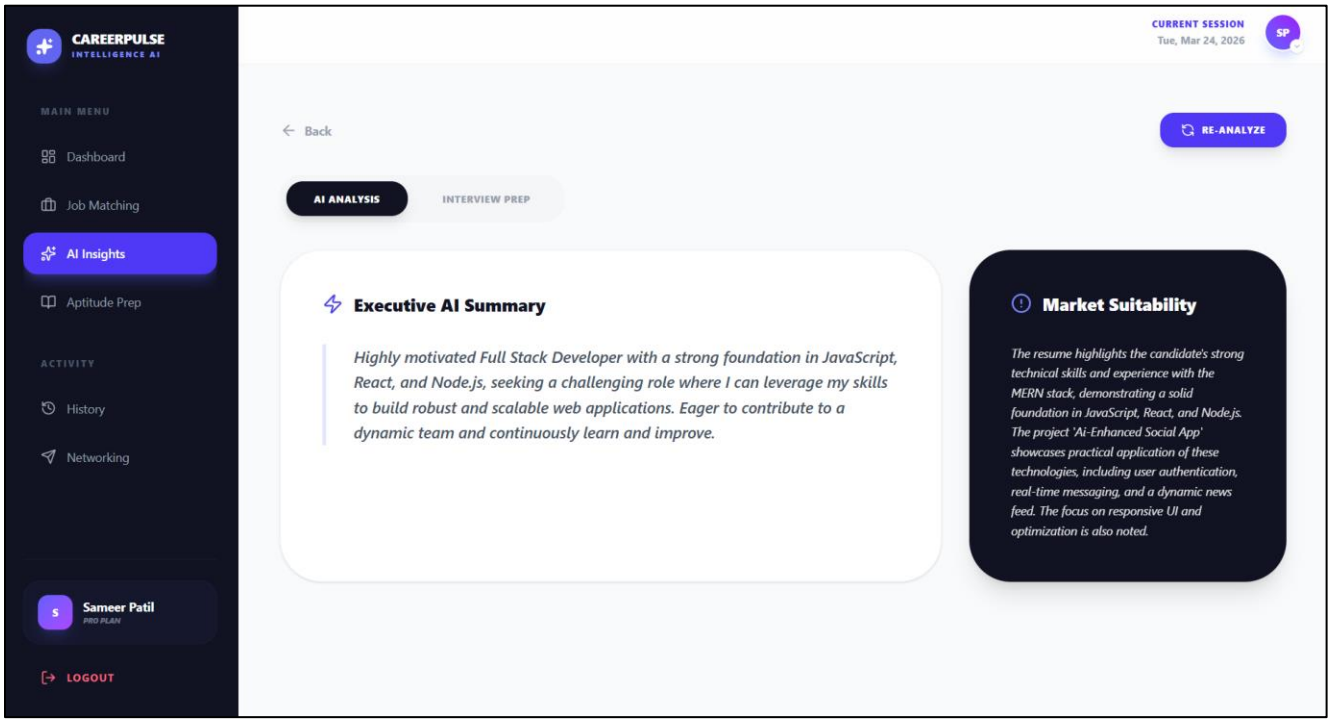


Fig.4.3.3:- Technical Summary Card:

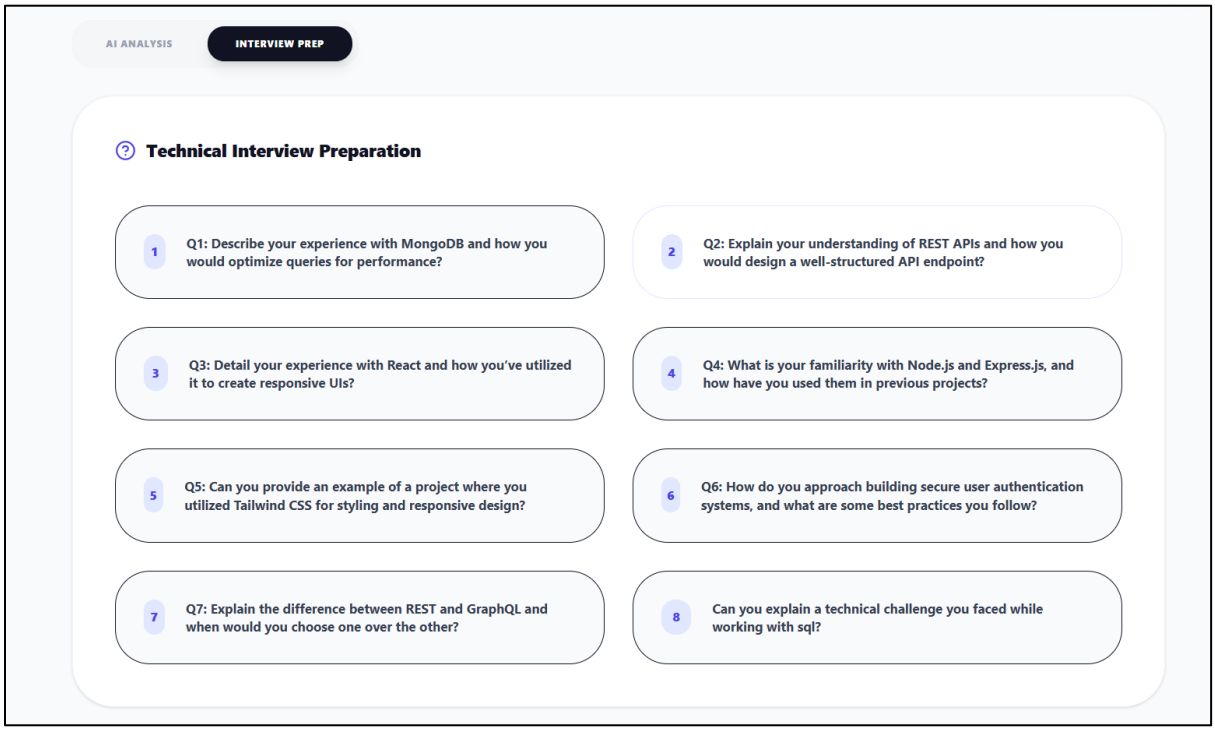


Fig.4.3.3:- Genrated question from local ai

4.3.4 The Job Matcher Matrix and Skill Roadmap

- **Layout Description:** Designed to provide immediate visual feedback on career readiness, this screen renders the output of the static jobMatch.service.js engine. It displays a responsive CSS grid of professional role cards (e.g., Frontend Developer, Database Administrator).
- **Color Coding:** Each role card features a progress bar representing the "Match Score %". Below the bar, the layout utilizes dynamic CSS color-coding: **Green highlighted tags** indicate possessed skills (e.g., React, Node), while **Red highlighted tags** emphasize missing technologies (e.g., Docker, Python). This visual distinction effectively serves as a personalized study roadmap for the student.

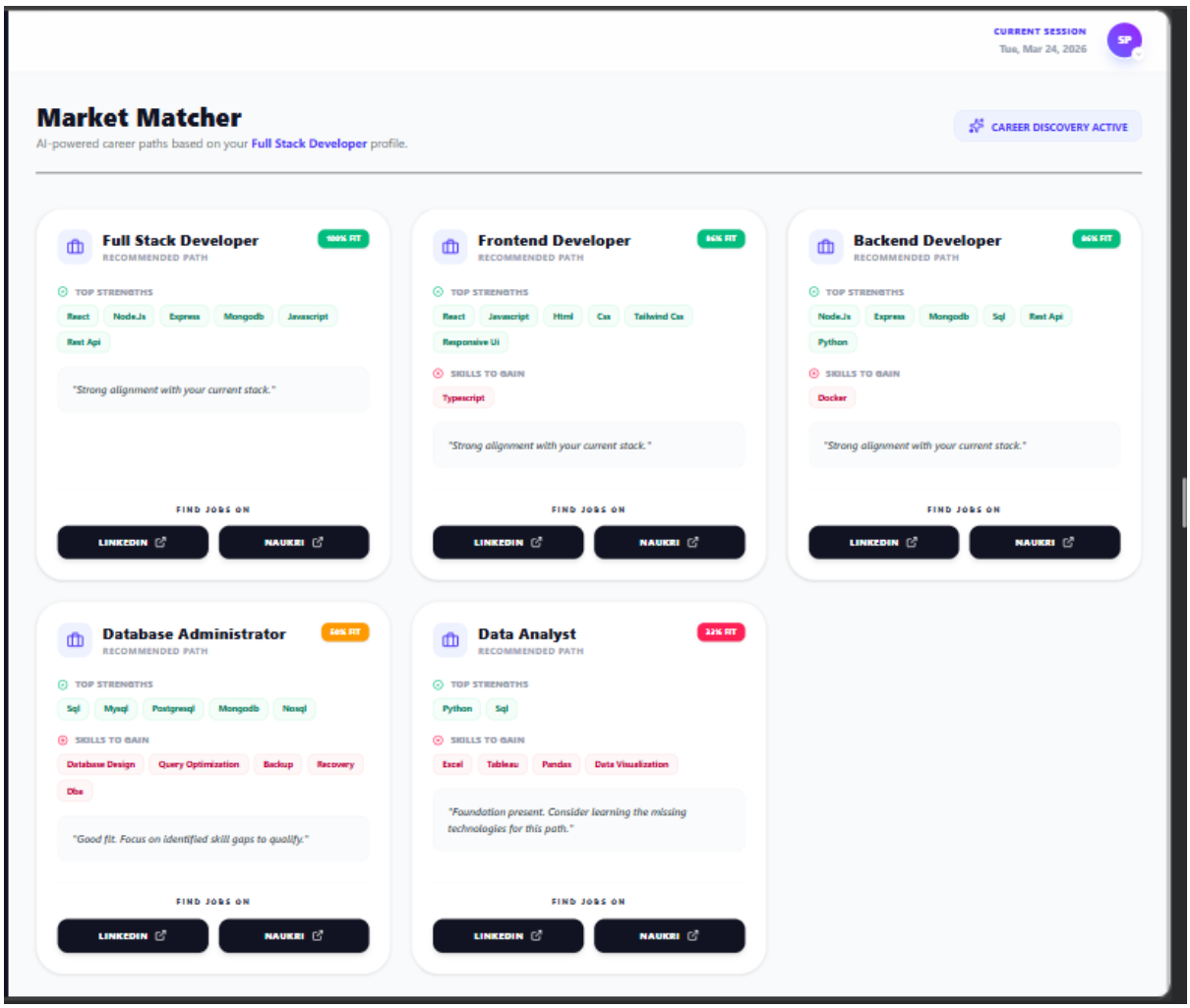


Fig.4.3.4:-Job matching page:-

4.3.5 The Placement Prep Hub (Aptitude Module)

- **Layout Description:** A dedicated educational interface built for technical and logical reasoning practice. The layout is vertically split; the left navigation panel allows users to select assessment categories (Quantitative Aptitude, Logical Reasoning, Coding Aptitude, Verbal Ability). The main display area serves randomized, multiple-choice questions pulled from the static questionPool array.
- **Interactivity:** As the user selects options, the React state immediately evaluates the choice. The UI provides instant feedback, turning the selected box green for correct answers or red

for incorrect ones, followed by a detailed mathematical or logical explanation of the solution to facilitate active learning.

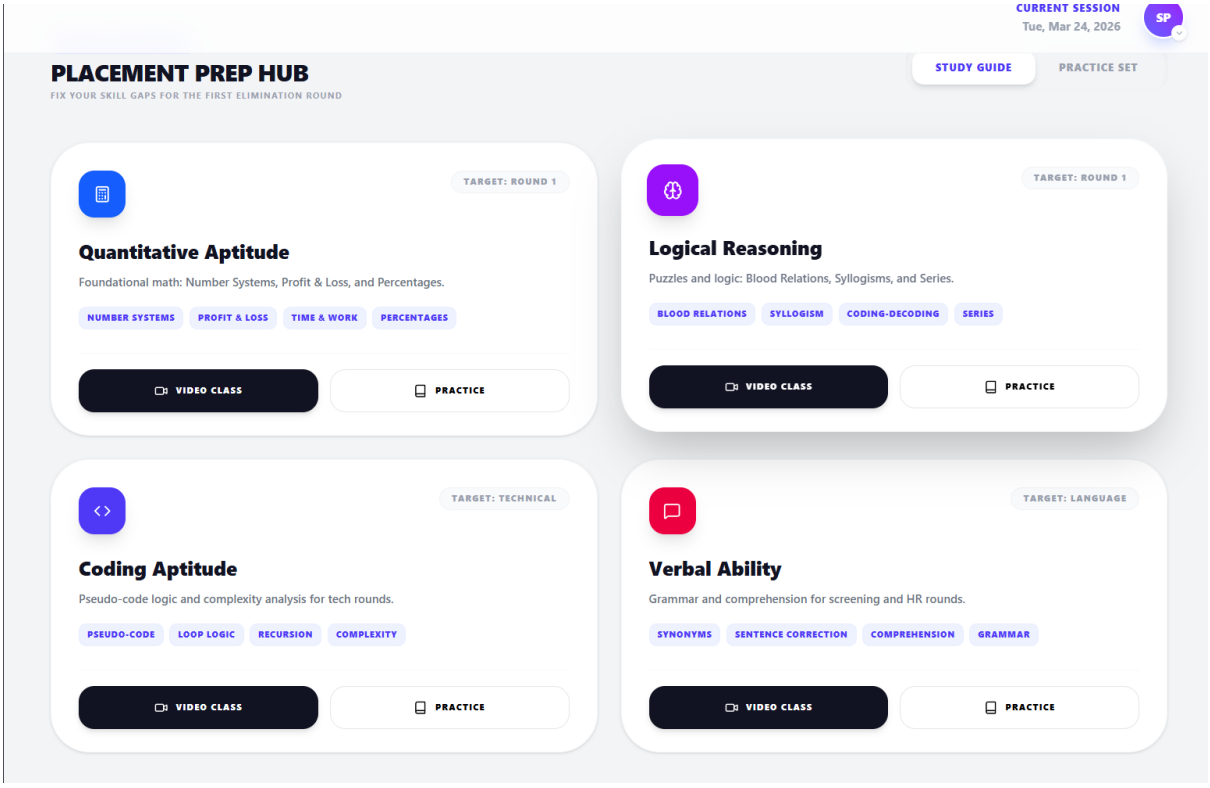


Fig.4.3.5:- Aptitude prepare page

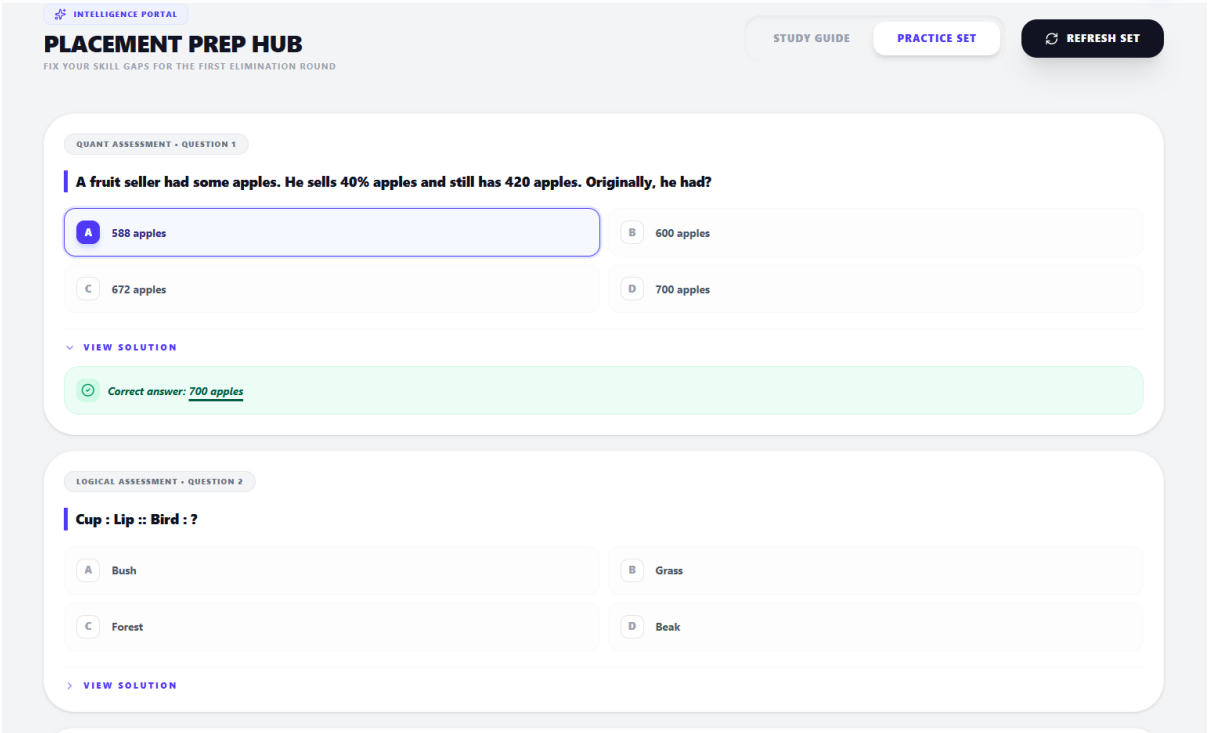


Fig.4.3.5:- Aptitude practice page

4.3.6 The User History Interface

- **Description:** This interface provides a chronological ledger of the student's previously analyzed resumes. It features a clean, data-dense table displaying original uploaded filenames, timestamps, computed ATS scores, and the AI-predicted professional roles.
- **Interactivity:** The table supports client-side sorting and filtering. Clicking any historical entry instantly retrieves the cached AI JSON report from MongoDB, allowing students to review past interview questions and career roadmaps without having to re-trigger the computationally expensive local LLM.

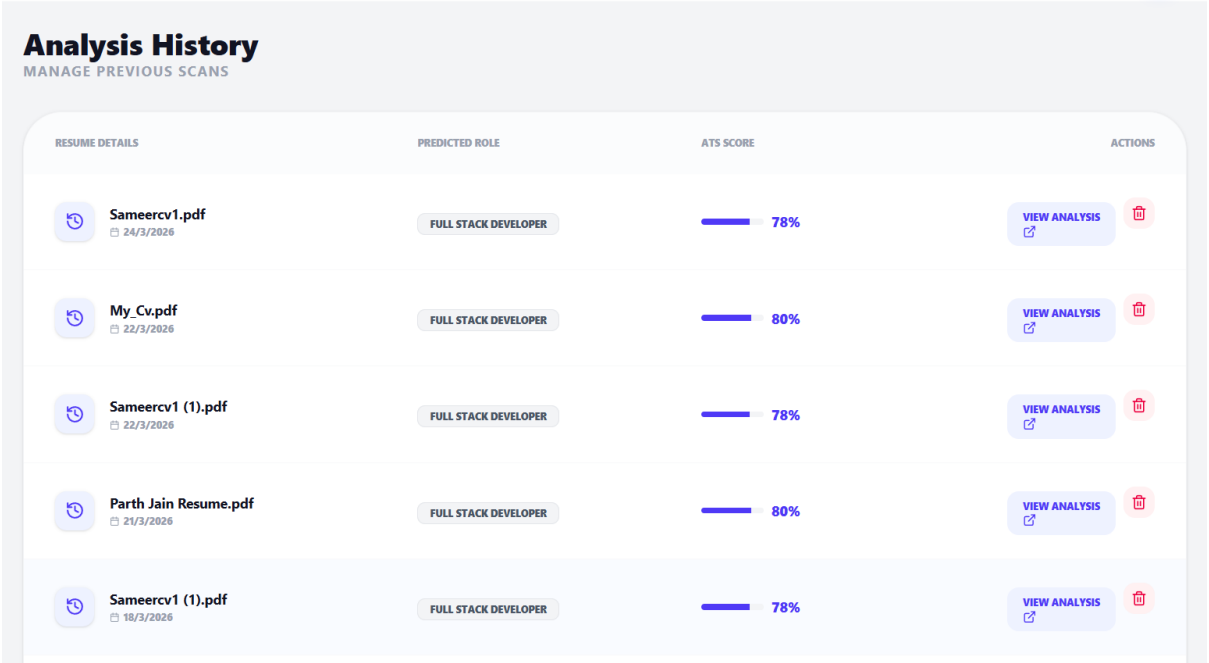


Fig.4.3.6

Summary

This chapter successfully transitioned the theoretical system architecture detailed in Chapter 3 into practical, implemented code. It explicitly highlighted the extensive custom, static algorithms developed for document validation, job matching, data sanitization, and fallback execution—proving that the system relies on robust software engineering rather than just AI APIs. Furthermore, it outlined the structural integrity of the MongoDB database and provided a comprehensive visual tour of the React frontend application.

Chapter 5

Analysis and Related Work

5.1 System Analysis

The CareerPulse AI project was comprehensively analyzed post-development to determine its operational effectiveness, reliability under load, and success in providing a transparent, secure environment for student career preparation. The system analysis focused heavily on three core technological pillars:

- **Architectural Efficiency and Concurrency:** The system's architecture was designed to handle high computational loads (inherent to LLM inference) without degrading the user experience. By utilizing the asynchronous nature of the Node.js event loop within the MERN stack, the backend achieves low-latency responses for standard CRUD operations. The intelligent integration of asynchronous background workers ensures that the frontend React interface remains highly interactive. While the local LLM processes the heavy technical reporting in the background over several seconds, the user immediately receives initial ATS scoring and basic skill detection results, ensuring a seamless, non-blocking UX.
- **AI Accuracy and Validation Analysis:** A critical aspect of the system's efficiency is preventing "garbage data" (e.g., uploading blank PDFs, syllabuses, or non-resume images) from consuming valuable AI processing power. The system was tested against various document formats to evaluate the "Ultra-Strict Rejection Logic." Analysis proves that the logic successfully filters out invalid files by rigorously checking for the intersection of at least three major professional sections (such as Education, Experience, and Projects), verifying standard contact regex patterns (Email/Phone patterns), and enforcing a strict 600-character minimum density threshold.
- **Privacy Sovereignty and Data Security:** Unlike dominant cloud-based competitors that mandate uploading personal documents to external servers, this project's analysis confirms absolute data sovereignty. By executing a local **Ollama (gemma3:1b)** instance directly on the host server, no Personally Identifiable Information (PII) or employment history is ever transmitted over the public internet to third-party API providers. This architectural decision perfectly fulfills the project's core motivation of ensuring uncompromising data privacy for vulnerable students.

5.2 Testing Methodology and Results

To ensure high reliability, fault tolerance, and academic rigor, the system underwent multiple layers of testing, mirroring industrial software quality assurance standards.

- **Unit Testing (Parsing Service):** Granular tests were conducted on the resumeParser.service to ensure precise text extraction from varying file structures. The service was tested using pdf-parse for standard PDFs and mammoth for DOCX files. The results confirmed that the system correctly identifies and throws structured error messages for empty files, corrupted buffers, or unsupported formats, preventing downstream backend crashes.
- **Integration Testing (AI Insights & Fallback Logic):** The sequential flow between the backend HTTP controller, the internal AIService, and the external local Ollama model was heavily tested to ensure consistent JSON output formatting. Because generative AI models can occasionally "hallucinate" or return malformed syntax, a robust JSON cleanup algorithm was tested. It successfully strips escaped quotes, stray newlines, and trailing commas. Furthermore, testing confirmed that the "Fallback Analysis" logic successfully intercepts complete AI failures, returning a standardized set of 8 generic but high-quality interview questions to ensure the frontend UI never breaks or displays empty screens.
- **Functional Testing (Job Match Engine):** The job matching service was subjected to functional testing by injecting resumes spanning a wide variety of tech stacks (e.g., MERN, Data Science, DevOps). The results indicated that the algorithm correctly identifies "missing skills" and provides an accurate, mathematically sound percentage match score by cross-referencing extracted skill arrays against twelve hardcoded, industry-standard role templates (such as Full Stack Developer, Cyber Security Analyst, etc.).

5.3 Related Work Done

The development of CareerPulse AI builds upon and significantly advances existing research in the domains of automated recruitment, natural language text classification, and educational technology.

- **Evolution of Natural Language Processing (NLP) in Hiring:** Previous academic and industrial works have extensively explored keyword-based filtering using statistical techniques like TF-IDF (Term Frequency-Inverse Document Frequency). However, this project advances the field by moving beyond simple keyword arrays. By utilizing Generative AI (gemma3:1b), the system contextually understands the relationship between

skills and projects. This allows for the dynamic generation of highly specific interview questions, a complex feature largely missing in traditional, rigid ATS research methodologies.

- **Local LLM Implementation & Edge Computing:** Recent research in privacy-preserving AI highlights a necessary paradigm shift toward "Edge AI" and localized inference to protect sensitive user data from corporate harvesting. CareerPulse AI serves as a highly practical implementation of this research by deploying the Ollama framework to execute complex natural language tasks purely on local hardware, effectively eliminating the latency, cost, and severe privacy concerns associated with API-driven models like ChatGPT.
- **Explainable AI (XAI) in ATS Optimization:** Existing sociological and technical literature heavily criticizes the "Black Box" nature of automated recruitment, where human biases are simply replaced by algorithmic opacity. This project actively contributes to the growing field of "Explainable AI" (XAI). Instead of delivering an unexplained, arbitrary numerical score, the system provides the student with a 9-10 line detailed journey and a market suitability explanation, effectively "opening the black box" and offering clear, actionable reasons for the system's technical evaluations.

5.4 Comparison with Existing Systems

A comprehensive comparative analysis was performed to benchmark CareerPulse AI against standard market solutions and corporate ATS platforms. The findings illustrate a clear operational advantage in terms of student empowerment, transparency, and financial accessibility.

Table 5.4

Feature Matrix	Standard ATS / Commercial Cloud Tools	CareerPulse AI (Proposed System)
Data Privacy & Sovereignty	Extremely Low (Cloud-based storage & data harvesting)	High (100% Local AI Processing; no data leaves the host)
Depth of Candidate Feedback	Generic numerical score (e.g., "75% Match")	Contextual 7-8 line technical journey & explanation
Interview Preparation	Not integrated (requires third-party tools)	Generates project-specific questions dynamically
Aptitude Practice	Requires fragmented platforms (e.g., HackerRank)	Integrated prep hub (Quant, Logical, Coding)
Financial Accessibility	Expensive subscriptions or pay-per-use	Zero cost (Open Source / Local Model)

Summary

This chapter provides a rigorous evaluation of the system's operational effectiveness, concurrency handling, and overall AI accuracy. It proves the platform's efficiency in executing heavy local LLM inference tasks seamlessly in the background without blocking the Node.js event loop or degrading the frontend user experience. The analysis confirms absolute data sovereignty, verifying that no personal data is transmitted to external cloud servers at any point during the resume screening process. It directly compares CareerPulse AI against commercial alternatives, highlighting its superior transparency and financial accessibility. It also documents real-world engineering challenges, detailing how the static fallback execution guarantees users still receive interview questions even if the AI fails.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

Building CareerPulse AI was ultimately about solving a very real problem that every IT student faces today: the frustrating, invisible wall of automated hiring software. By taking this project from a basic idea to a fully functioning MERN-stack web application, we successfully tackled the exact issues that make applying for jobs so difficult. Looking at the final working platform, there are a few major technical and practical takeaways we can confidently draw from the development process:

- **Fixing the "Black Box" of Hiring:** We successfully proved that it is possible to give students actual transparency into how their resumes are graded. Instead of just handing them a random ATS score without context, the system writes out a custom technical summary that explains exactly how a machine reads their profile. This means students no longer have to guess why they got rejected; they have clear, actionable data telling them what to fix before they apply for their next job.
- **Real Data Privacy Using Local AI:** One of the biggest technical wins of this project was getting a Large Language Model to run locally on our own backend. By using Ollama with the gemma3:1b model, we proved that you don't need to send sensitive student data to paid cloud APIs like OpenAI. We built a completely private system where a user's personal information, phone numbers, and job history never leave the local server, making it far more secure than the commercial tools currently out there.
- **Building an All-In-One Practice Hub:** We didn't want to stop at just fixing resumes, because passing the ATS is only the first step. By coding an interactive Placement Prep Hub directly into the React frontend, we managed to put everything a student needs in one single place. Whether they need to practice coding questions, logical reasoning, or quantitative math, they can do it all from one dashboard instead of jumping between five different preparation websites.
- **Smart Job Matching and Skill Tracking:** The custom Node.js backend algorithm we wrote actually compares the skills pulled from the resume against real-world job roles, like Full Stack Developer or DevOps Engineer. It works exactly as intended, explicitly telling the user what technologies they are missing. This turns a boring, static PDF into an actual learning roadmap that tells a student what course they should take next to become highly employable.

6.2 Future Work

Even though CareerPulse AI is fully functional and ready to use, software can always be improved. During the development and testing phases, we came up with several exciting ideas for how this platform could be expanded in the future to make it even more powerful for job seekers:

- **Live Audio Mock Interviews:** Right now, the AI generates eight custom interview questions based on the user's projects. In the future, we could add Web Speech API integrations (speech-to-text and text-to-speech) so the browser actually talks to the student. This would allow them to practice answering technical questions out loud, and the AI could grade their spoken answers for confidence and accuracy.
- **Upgrading to Larger AI Models:** We used the lightweight gemma3:1b model so it could run smoothly on standard computers. As server hardware gets better and cheaper, a future update could swap this out for a much larger model, like Llama 3 8B or Mistral. This would give even smarter technical summaries, fewer parsing mistakes, and allow the system to handle complex, back-and-forth conversations with the user.
- **Connecting to GitHub and LinkedIn:** It is easy for a student to stretch the truth on a PDF resume. A great future feature would be adding secure API logins (OAuth) for GitHub and LinkedIn. The platform could automatically check a student's actual GitHub commit history to prove to recruiters that they really do know how to code in React or Python, cross-verifying what they claimed on paper.
- **Automated Cold Email Writing:** Reaching out to recruiters on LinkedIn or via email is exhausting. Since our AI already knows everything about the student's skills and projects, we could easily add a button that automatically writes a highly professional, custom cold email. The student could just paste in a job description, and the AI would write the perfect pitch for them to send.
- **Better Dashboard Analytics and Graphs:** We could upgrade the React frontend to include advanced data visualization libraries, like Chart.js or D3.js. This would let us add radar charts to show a student's skill balance, or line graphs that visually track how their ATS score has improved over the last six months of uploading new resumes.

6.3 References

The following official documentation, libraries, and educational portals were utilized as the primary technical references for the successful development and implementation of the CareerPulse AI system

1. React.js Official Documentation Served as the primary guide for building the interactive, state-driven frontend Single Page Application (SPA) and managing React Hooks (useState, useEffect) for real-time dashboard updates. Available at: https://react.dev/

2. Node.js and Express.js API Reference Provided the architectural guidelines for designing the asynchronous backend server, managing event loops, and establishing secure RESTful API endpoints for resume processing. Node.js Docs: https://nodejs.org/en/docs/ Express.js Docs: https://expressjs.com/

3. MongoDB and Mongoose Documentation Essential for designing NoSQL database schemas, connecting the backend to MongoDB Atlas, and handling complex nested JSON queries for AI-generated reports. MongoDB Docs: https://www.mongodb.com/docs/ Mongoose Docs: https://mongoosejs.com/docs/

4. Ollama Open-Source Framework Utilized for understanding the integration of local Large Language Models (LLMs), pulling the gemma3:1b model, and managing HTTP requests to the local inference engine. Available at: https://ollama.com/

5. Tailwind CSS Documentation Served as the reference for applying utility-first CSS classes to design a responsive, modern, and high-contrast user interface across the placement portal. Available at: https://tailwindcss.com/docs

6. GeeksforGeeks (GFG) Acted as a core educational reference for structuring the coding aptitude questions and verifying the logic for the technical interview preparation module. Available at: https://www.geeksforgeeks.org/

7. IndiaBix Educational Portal Used as a reference point for curating the high-quality quantitative, logical reasoning, and verbal ability question datasets utilized in the Placement Prep Hub. Available at: https://www.indiabix.com/

8. PDF-Parse Library Repository (NPM) Provided the technical implementation logic required for extracting raw, readable text strings from binary PDF document files uploaded by the user. Available at: https://www.npmjs.com/package/pdf-parse

9. JSON Web Tokens (JWT) Documentation Used as the official security reference for implementing stateless, cryptographic user authentication and securing private backend middleware routes. Available at: https://jwt.io/

10. Multer Middleware Documentation (NPM) Served as the technical guide for safely intercepting multipart/form-data requests, configuring disk storage, and validating file types during resume uploads. Available at: https://www.npmjs.com/package/multer

11. OWASP Top Ten (Security Reference) Essential for understanding and preventing common web vulnerabilities like SQL injection and Cross-Site Scripting (XSS), ensuring the CareerPulse AI authentication flow is enterprise-grade. Available at: https://owasp.org/www-project-top-ten/

12. Postman API Documentation & Testing Guide Utilized as the primary tool for testing RESTful API endpoints, verifying JSON response structures, and debugging the asynchronous communication between the Node.js backend and the local Ollama engine. Available at: https://learning.postman.com/docs/publishing-your-api/documenting-your-api/

13. MDN Web Docs (Mozilla Developer Network) The core technical reference for implementing modern JavaScript (ES6+) features, managing asynchronous Promises for AI processing, and optimizing the browser's Fetch API. Available at: https://developer.mozilla.org/en-US/

14. Axios Library Documentation (NPM) Served as the technical reference for handling promise-based HTTP requests between the React frontend and the Express backend, specifically for managing file upload progress bars. Available at: https://axios-http.com/docs/intro

15. Google Lighthouse Performance Metrics Used as a benchmarking tool to analyze and optimize the React frontend's "Largest Contentful Paint" (LCP) and accessibility scores, ensuring the dashboard remains fast and responsive. Available at: https://developer.chrome.com/docs/lighthouse/overview/